A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

Sofia Gabrielle I. Belarmino
Bachelor of Science in Computer Science

Lester Liby V. Ojeñar
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Ian Peter L. Lastimosa
Jelly P. Aureus, MS
Jerome G. Flores
Jasper V. Gomez

September 2026
Naga City, Philippines

Keywords: graph neural networks, multi-label classification, set learning

Copyright 2026, Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

The Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Jelly P. Aureus, MS
Panel Member
Date signed: _____

Jerome G. Flores
Panel Member
Date signed: _____

Jasper V. Gomez
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Project Advisor
Date signed: _____

The Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Jelly P. Aureus, MS

Chair, Department of Computer Science

Date signed: _____

Marianne A. Tolentino, MS

Dean, College of Computer Studies

Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

Sofia Gabrielle I. Belarmino
4 - Bachelor of Science in Computer Science

Lester Liby V. Ojeñar
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Ian Peter L. Lastimosa
Project Advisor

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

by

Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

Project Advisor: Ian Peter L. Lastimosa

Department of Computer Science

ABSTRACT

Combining two molecular representations is an unordered-pair learning problem: exchanging the two inputs should not change the prediction, while the model may still need to preserve relationships between them. This study will compare mean pooling, order-averaged concatenation, and Set Transformer aggregation for multi-label prediction of catalog-recorded descriptors of two-molecule blends. The three variants will use a shared molecular graph encoder and matched training conditions. Order-averaged concatenation will be treated as an adapted baseline based on a published pairwise concatenation design, not as a full reproduction of that model.

The catalog descriptors will be treated as proxy targets rather than controlled sensory-panel measurements. Reliability-aware eligibility masking and component-disjoint partitioning will be used as evaluation controls: the first will account for uncertain catalog annotations, while the second will reduce repeated molecular components across data partitions. The models will be compared on reliability-screened test cells using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics. Additional analyses will examine catalog-label patterns and how the relative performance of the three methods changes when training-label reliability is reduced, including under controlled label corruption. Numerical results will be reported only after the finalized evaluation pipeline has been run.

TABLE OF CONTENTS

1 Introduction **1**

1.1 Project Context 2

1.2 Purpose and Description 3

1.3 Objectives 5

1.4 Scope and Limitations 5

1.5 Definition of Terms 7

2 Review of Related Literature **9**

2.1 Computational Olfaction and Single-Molecule Prediction 9

2.1.1 History of QSOR Modeling 10

2.1.2 The Principal Odor Map 10

2.1.3 Limitations of Single-Molecule Models 11

2.2 Psychophysics of Odor Mixtures 11

2.2.1 Scent Blending Mechanics 12

2.2.2 Evidence That Averaging May Be Sufficient 12

2.3 Machine Learning on Sets and Chemical Mixtures 13

2.3.1 Permutation Invariance and Deep Sets 13

2.3.2 Set Transformer (SAB and PMA) 14

2.3.3 Three Ways to Combine Two Vectors 14

2.3.4 Precedents in Mixture Modeling 15

2.4 Olfactory Datasets and Label Reliability 16

2.4.1 Reference Catalogs 16

2.4.2 The Blended-Pair Dataset . . . 16
2.4.3 Human-Rated Datasets . . . 17
2.4.4 Absence of a Standard Vocabulary . . . 17
2.4.5 Noisy and Imbalanced Labels in Multi-Label Learning . . . 17
2.5 Out-of-Distribution Splitting Strategies . . . 18
2.6 Evaluating and Comparing Models Under Label Sparsity . . . 19
2.7 Synthesis and Research Gap . . . 20
2.7.1 Synthesis of Literature . . . 20
2.7.2 Statement of the Research Gap . . . 21
2.7.3 Positioning of the Present Study . . . 21

3 Theoretical and Conceptual Framework 22
3.1 Theoretical Framework . . . 22
3.1.1 Representing Molecules as Graphs . . . 22
3.1.2 Graph Neural Networks and Message Passing . . . 23
3.1.3 Graph Attention Encoder . . . 24
3.1.4 Combining an Unordered Pair . . . 26
3.1.5 Representation Trade-offs . . . 27
3.1.6 Sparse Labels and Label Reliability . . . 28
3.2 Conceptual Framework . . . 30
3.2.1 Input Phase . . . 30
3.2.2 Process Phase 1: Data Preparation and Graph Carving . . . 31
3.2.3 Process Phase 2: Encoding and Pair Representation . . . 31
3.2.4 Process Phase 3: Classification and Optimization . . . 32
3.2.5 Output Phase . . . 32
3.3 Technical Background . . . 33
3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization . . . 33
3.3.2 Deep Learning Stack: Sparse Graph Convolutions . . . 34
3.3.3 Multi-Label Formulation: Sigmoids and Sparsity Mitigation . . . 34
3.3.4 Evaluation Metrics for Sparse Multi-Label Outputs . . . 35

4 Methodology **37**

4.1 Research Design . . . 37

4.2 Data Acquisition and Preparation . . . 38

4.2.1 Data Sources and Provenance . . . 38

4.2.2 Catalog Targets and Their Limits . . . 41

4.2.3 Target Vocabulary . . . 41

4.3 Label Reliability Mask . . . 42

4.3.1 Consistency Evidence . . . 42

4.3.2 Eligibility Mask . . . 43

4.3.3 Experimental Label Conditions . . . 43

4.4 Model Architecture . . . 44

4.4.1 Shared Graph Encoder . . . 44

4.4.2 Aggregation Variants . . . 45

4.4.3 Output Head and Loss . . . 46

4.5 Component-Disjoint Partitioning and Graph Carving . . . 47

4.5.1 Scaffold Split Decay . . . 47

4.5.2 Cluster-Based Partitions . . . 48

4.5.3 The Graph-Carving Protocol . . . 48

4.5.4 Training-Only Vocabulary Filter . . . 49

4.6 Experiments . . . 49

4.6.1 Experiments A and B: Main Comparison (RQ1) . . . 49

4.6.2 Experiment C: Controlled Label Corruption (RQ3) . . . 50

4.6.3 Blend-Type Analysis (RQ2) . . . 50

4.7 Evaluation Metrics . . . 51

4.7.1 Multi-Label Metrics . . . 51

4.7.2 Masked Metrics . . . 52

4.8 Statistical Comparison . . . 52

4.9 Reporting Conventions . . . 53

A Chemical Feature Vectors **55**

A.1 Symmetry Collapse: Representation-Theoretic Derivation . . . 57

A.2 Dataset Profiles and Canonical Vocabulary . . . 58

A.2.1 Single-Molecule Harmonized Dataset Sample 58
A.2.2 Pairwise Blend Mixture Sample 58
A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate 58

LIST OF FIGURES

3.1 Input-Process-Output (IPO) Model. 30

4.1 Compact overview of the proposed computational pipeline. The three aggregation branches receive the same frozen molecular embeddings and use the same prediction and evaluation protocol. 39

LIST OF TABLES

3.1 Aggregation methods for an unordered pair of molecule embeddings. 26
4.1 Varied factors and matched controls. 38
4.2 Cell-level rules for the eligibility mask. 43
4.3 The three aggregation variants. 45
A.1 Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i). 56
A.2 Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}). 56
A.3 Representative sample of harmonized single odorants. 58
A.4 Sample pairwise odor mixtures from the Sisson et al. benchmark with catalog-recorded descriptors. 58
A.5 The 104 standardized odor descriptors (alphabetical). 59

Chapter 1

Introduction

Combining two graph-derived vectors is an unordered-pair learning problem. Each molecule can be encoded as a graph and represented by a learned vector, but a blend should produce the same prediction whether the inputs are presented as (A, B) or (B, A). The combination method must therefore preserve input-order invariance while retaining information about the particular pair being modeled. Mean pooling is a simple order-independent baseline, but it can map different pairs to the same representation: the pairs $(2, 8)$ and $(5, 5)$ have the same average even though their elements differ. More expressive aggregation may preserve distinctions that averaging loses, but it also adds parameters and computation. Whether that added complexity produces a useful improvement is an empirical question, not an assumption of this study [41].

Fragrance-blend prediction provides the application setting for this pairwise learning problem. Graph Neural Networks (GNNs) have been used to learn molecular representations directly from structure, including in single-molecule odor prediction [23, 27]. Extending a single-molecule encoder to a two-molecule input requires a separate aggregation design. This study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for predicting catalog-recorded descriptors of binary fragrance blends [24, 37].

The blend descriptors used for training and evaluation are catalog records, not direct measurements of human perception. Public blend datasets such as the one compiled by Sisson et al. [37] are based on reference-catalog recommendations rather than a controlled sensory panel. The study therefore measures label consistency using reciprocal and cross-catalog agreement before comparing

the aggregation methods, and reports performance under declared reliability conditions rather than treating every catalog label as unquestioned ground truth.

1.1 Project Context

The fragrance domain supplies a useful application for studying this representation problem. Existing molecular GNNs can encode individual structures, but a pairwise blend task requires a second design decision: how to combine two molecule embeddings into one order-independent representation before multi-label classification [23, 37]. The unresolved issue addressed here is therefore not computational olfaction in general. It is the controlled comparison of alternative pair-combination methods under the same encoder, training conditions, and evaluation protocol.

The data structure also creates a partitioning problem. A blend dataset is organized as pair records, and the same molecule can occur in many different pairs. A random split of pair records can therefore place one molecule in both training and test partitions, allowing the model to reuse a component it has already seen. This study uses a component-disjoint split and graph-carving procedure to reduce that overlap by excluding pairs that cross the assigned partitions. The procedure is intended to reduce component leakage; it does not guarantee that every held-out molecule belongs to an entirely unfamiliar chemical family [12].

The labels create a second evaluation problem. Each blend may have several descriptors, but only a small fraction of the possible descriptor cells are positive. A model can therefore obtain a deceptively favorable aggregate score by predicting mostly absent labels, while inconsistent catalog records can change the apparent ranking of the aggregation methods. The study addresses these risks through multi-label metrics suited to sparse outputs and through a reliability audit based on reciprocal listing and cross-catalog agreement. The audit is used to define declared reliability conditions; it does not convert catalog records into verified measurements of human perception.

Fragrance and catalog terminology are retained only to identify the input domain and target descriptors. The study does not attempt to infer receptor mechanisms, explain physical mixture chemistry, or model perfume-formulation practice. Its computer-science focus is the representation and evaluation of an unordered pair of molecular graphs.

1.2 Purpose and Description

This study contributes a controlled empirical comparison of mean pooling, order-averaged concatenation, and Set Transformer aggregation under a shared molecular-graph encoding framework. It does not propose a new neural architecture. Instead, it evaluates whether an existing attention-based set aggregator provides a measurable benefit over simpler alternatives for predicting catalog-recorded descriptors of two-molecule blends [24, 37].

In all three variants, a Graph Attention Network (GAT) encodes each molecular graph into a learned vector using the same architectural specification and initialization policy [49]. The experimental variants differ in how the two molecule vectors are combined. Mean pooling averages the vectors directly. Order-averaged concatenation evaluates both input orders and averages their outputs. Set Transformer aggregation uses Self-Attention Blocks (SAB) and Pooling by Multihead Attention (PMA) to learn statistical relationships between the two vectors before producing an order-independent blend representation [24, 51].

A shared multi-label classification design maps each combined representation to independent sigmoid outputs for the target catalog descriptors. The variants are compared using matched data partitions, training conditions, and evaluation metrics. Reliability-aware evaluation reduces the influence of uncertain catalog annotations, while component-disjoint evaluation reduces the risk that the same molecular component appears in both training and test pairs. These controls strengthen the comparison, but they do not establish label correctness, eliminate all structural similarity, or guarantee that the Set Transformer will outperform the simpler methods.

Significance of the Study

The study is primarily intended to contribute evidence for computer-science research on pairwise representation learning and model evaluation. Its findings may be useful to the following groups:

- **Machine-Learning and Graph-Representation Researchers.** The study provides a controlled comparison of three methods for combining an unordered pair of graph embeddings. It contributes evidence on whether an attention-based set aggregator offers a measurable benefit over simpler methods, how model comparisons change under uncertain labels, and how component-disjoint splitting can reduce the influence of repeated molecular components across data partitions. Computational olfaction serves as the test domain for this broader represen-

tation and evaluation problem.

- **Researchers and Maintainers of Olfactory Reference Catalogs.** The reciprocal and cross-catalog agreement analyses provide a way to describe the consistency of catalog-derived labels before they are used for machine learning. These analyses may help identify descriptors that require closer review, although they do not determine whether a catalog label is perceptually correct.

- **Fragrance Researchers and Practitioners.** The results may provide preliminary evidence about the strengths and limits of structure-based prediction for catalog-recorded blend descriptors. However, this study does not validate a formulation tool or measure reductions in laboratory work, material use, or trial-and-error blending [31].

Research Questions

This study seeks to answer the following questions:

RQ1. How do mean pooling, order-averaged concatenation, and Set Transformer aggregation compare in predicting reliable catalog-recorded descriptors of two-molecule blends under the same training and evaluation conditions?

RQ2. How does the performance of the three aggregation methods differ across additive, catalog-label loss, and catalog-label gain groups?

RQ3. How does changing the reliability of the training labels, including through controlled label corruption, affect the performance of the three aggregation methods?

The study also has a limited connection to the United Nations Sustainable Development Goals. Its work on computational methods and data-quality evaluation is most closely related to SDG 9. Structure-based prediction could eventually support tools related to responsible material use under SDG 12, but the present study does not measure material savings, reduced laboratory work, or reduced trial-and-error formulation. These are possible future applications, not outcomes evaluated in this research.

1.3 Objectives

General Objective

To compare mean pooling, order-averaged concatenation, and Set Transformer aggregation as methods for combining an unordered pair of molecular graph embeddings for multi-label prediction of catalog-recorded blend descriptors, using matched training conditions and a reliability-aware, component-disjoint evaluation protocol.

To achieve the general objective, the following specific objectives must be accomplished:

- To prepare the single-molecule and blend datasets and describe the consistency and sparsity of their catalog-recorded labels.
- To construct fixed component-disjoint data partitions and reliability-screened evaluation conditions for comparing the models.
- To implement mean pooling, order-averaged concatenation, and Set Transformer aggregation using the same GAT encoder design, output task, loss function, and matched training protocol.
- To compare the three aggregation methods using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics.
- To determine how the relative performance of the three methods changes across label-reliability conditions, controlled label corruption, and additive, catalog-label loss, and catalog-label gain patterns.

1.4 Scope and Limitations

This study is limited to pairwise, two-molecule blends and the prediction of catalog-recorded descriptors. Each molecule is represented as a two-dimensional molecular graph and encoded by a shared Graph Attention Network (GAT) [49]. The study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation as three methods for combining the resulting molecule embeddings. The Set Transformer uses attention to learn statistical relationships between the embeddings; it is not treated as a model of receptor activity or physical molecular interaction [24, 51].

The data are obtained from the GoodScents and Leffingwell single-molecule catalogs and the catalog-derived binary-blend dataset of Sisson et al. [7, 37]. The target descriptors are binary catalog records rather than intensity ratings or measurements from a newly conducted sensory panel. The study does not recruit human assessors and does not claim that its predictions represent what every person would perceive. Concentration, mixing ratio, and changes over time are also outside the scope; each blend is treated as an unweighted pair.

Multi-label learning requires both positive and negative targets. A descriptor recorded for a blend is treated as positive, while an unlisted descriptor is treated as negative under a closed-world assumption. This is an operational definition for model training and evaluation. Catalog omission may reflect an unrecorded observation rather than true sensory absence, so an unlisted cell is not treated as proof that the descriptor cannot be perceived.

Label reliability is assessed using reciprocal listing and cross-catalog agreement. These checks measure the consistency of the available records, not their perceptual truth. Reliability-screened training and evaluation use a cell-level eligibility mask: reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded. This procedure can reduce the influence of uncertain annotations, but it cannot remove every form of catalog bias or labeling error.

The models are evaluated on fixed component-disjoint partitions produced by the clustering and graph-carving procedure described in Chapter 4. Pairs that cross partition boundaries are excluded so that a retained training pair and a retained test pair do not share the same molecule. This design reduces component leakage, but it does not guarantee that the partitions have no structural similarity or that the test molecules represent every possible unseen chemical family [12].

The comparison is limited to the declared model variants, metrics, reliability conditions, and controlled label-corruption experiments. The results therefore show how these methods behave on the selected catalog-derived task and should not be generalized directly to larger mixtures, other descriptor vocabularies, or human sensory judgments. Order-averaged concatenation is an adapted baseline based on the pairwise design of Sisson et al.; it is not a full reproduction of that study's model. Final dataset sizes, target-vocabulary size, and label density will be reported only after the revised data pipeline has been run.

1.5 Definition of Terms

Molecular Graph

A computational representation of a molecule in which atoms are nodes, bonds are edges, and both carry features used by the graph encoder.

Graph Embedding

A fixed-length numerical vector produced by the graph encoder to summarize the learned information from one molecular graph.

Multi-Label Classification

A prediction task in which one blend can be assigned several descriptors at the same time. Each target descriptor is predicted separately as present or absent.

Mean Pooling

A combination method that takes the element-wise average of the two molecule embeddings. It is naturally independent of input order, but different pairs can produce the same average.

Order-Averaged Concatenation

A combination method that places the two molecule embeddings side by side, evaluates both input orders, and averages the two model outputs so that the final prediction does not depend on which molecule is listed first.

Set Transformer Aggregation

An attention-based method for combining an unordered collection of embeddings. In this study, it learns statistical relationships between the two molecule embeddings before pooling them into one blend representation.

Permutation or Order Invariance

The requirement that exchanging the positions of the two input molecules must not change the model's final prediction.

Component-Disjoint Split

A data partition in which the same molecule does not occur in both a retained training pair and a retained test pair. It is designed to reduce component leakage, but it does not guarantee that the

partitions contain no structurally similar molecules.

Reliability Mask

A cell-level indicator that determines which blend–descriptor targets are included in reliability-screened training or evaluation. Reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded.

Evaluation Metrics

Macro F1 gives each retained descriptor equal weight when averaging its F1 score. Micro F1 combines all evaluated decisions before computing one F1 score. Precision–Recall Area Under the Curve (PR-AUC) measures how well the model ranks positive targets across decision thresholds. Hamming loss reports the fraction of evaluated present-or-absent decisions that are incorrect.

Chapter 2

Review of Related Literature

This chapter reviews papers that help build the idea for this study. It begins with graph-based scent prediction for single molecules, then summarizes what is known about how odor mixtures behave to be able to explain the inputs and the labels this study has. Next, it discusses how an unordered pair of learned vectors can be combined, which covers the three methods this paper will be comparing: mean pooling, order-averaged concatenation, and Set Transformer aggregation, together with the precedents in mixture modeling. The chapter then reviews the catalog-derived datasets and talks about the problems regarding this study's labels, particularly their reliability, the noise, and their sparsity. This is followed by the data-splitting and model-comparison methods that help make evaluation credible. It ends by synthesizing the literature into the controlled-evaluation gap that this thesis addresses.

2.1 Computational Olfaction and Single-Molecule Prediction

This first section explores the evolution of computational olfaction from early QSOR modeling with manual feature engineering to more modern graph-based methods. It reviews how these frameworks were used in predicting scent from a single molecule and the limitations of such when applied to the multi-molecule blend prediction task this thesis aims to achieve.

2.1.1 History of QSOR Modeling

Quantitative Structure-Odor Relationship (QSOR) modeling refers to the prediction of the odors and olfactory properties of molecules through the characterization of their structure [14, 47, 52]. QSOR is especially important in fields such as the fragrance and flavor industries, as it could help reduce the trial-and-error involved in the formulation and development of new compounds computationally, as experimental wet lab methods can be both time-consuming and expensive [19, 33].

Early approaches in QSOR involved the manual creation of numerical features that describe and quantify the properties of a molecule [3, 14, 53]. These included molecular fingerprints, which encode the presence or absence of substructures in a binary vector, and physicochemical descriptors computed with dedicated software [3, 10, 35, 46].

Many studies pair such descriptors or fingerprints with classical learners such as random forest, XGBoost, and LightGBM to predict odor [33, 40, 45]. Two comparisons are representative. Suh et al. found that Morgan-fingerprint models outperformed descriptor-based ones on a curated set of 8,681 fragrance compounds [40], and Achebouche et al. found that graph convolutional representations outperformed general molecular descriptors and fingerprints on a set of 5,955 molecules [1].

While effective, these methods require the features to be chosen in advance [4, 23, 53], so the learning model works from a fixed summary of each molecule. Graph neural networks address this by learning, from the atoms and bonds themselves, which combinations of features are most useful for the task, although people still choose the input features and the network design [2, 4, 17, 52]. The best-known graph-based odor model is discussed next.

2.1.2 The Principal Odor Map

Addressing the aforementioned limitation is Lee et al.'s Principal Odor Map (POM), which maps a molecule's chemical structure to its smell using Graph Neural Network (GNN) techniques [23]. Rather than evaluating a molecule's scent using a static representation built from predefined hand-crafted descriptors, POM learns directly from the molecule's structure during training. It does this by treating each molecule as a graph, where nodes and edges represent atoms and chemical bonds, respectively; each node and edge is itself a vector, with nodes storing properties such as atomic number, and edges storing bond properties. Through message passing, the GNN dynamically assigns weights to each of the molecule's structural features, producing a numerical representation of the

molecule [23].

The setup was as follows: Lee et al. used approximately 5,000 molecules derived from merging data from the GoodScents and Leffingwell fragrance databases to produce a multi-label output of 138 distinct semantic odor descriptors. It was then tested on new molecules against a human benchmark, wherein approximately 15 trained panelists rated molecules using a narrowed-down set of 55 odor descriptors. The model's predictions fell within the range of individual panelists' ratings for every label except musk, and surpassed the median panelist on 30 out of the 55 labels (55%) [23]. This study differs from theirs because they checked their model against a newly collected human panel, while this study has no panel and evaluates against catalog-recorded labels only.

This shows that a graph model trained on catalog labels can predict panel-rated scent from molecular structure alone at roughly the level of a typical panelist. However, despite this significant advancement, these results were derived exclusively from single, unmixed molecules.

2.1.3 Limitations of Single-Molecule Models

POM shows what a graph encoder can learn for one molecule, but it operates on a single molecule at a time, while odorants rarely appear in isolation [8, 23, 43, 50]. In the real world, scents are rarely composed of single molecules and are instead blends that can contain individual components numbering up to the hundreds [16, 50]. A single-graph encoder produces one vector per molecule but it does not define how the vectors of two molecules should be combined into one prediction. That combining step is the subject of this study.

Lee et al. admit this limitation themselves, noting that while POM uses a coordinate system to categorize scents, it alone cannot determine the behavior and outcomes of different molecules when mixed [23].

2.2 Psychophysics of Odor Mixtures

Because single-molecule models do not specify how two molecules are combined, a method for combining them is needed. This section establishes only that both simple averaging and learned aggregation are plausible candidates for that method. It reviews how catalog blend labels differ from the labels of their parent molecules, and the human-panel evidence that averaging may be sufficient. It does not claim that either method reproduces perception.

2.2.1 Scent Blending Mechanics

A paper released by Sisson et al. introduces a labeled dataset consisting of molecule pairs and compares each pair's catalog-recorded blend descriptors with the descriptors of its parent molecules using two baselines: union and intersection. The union score assumes that the blend record contains any descriptor listed for either parent (additive labeling). The intersection score assumes that the blend record contains only descriptors shared by both parents (subtractive labeling) [37].

The Jaccard results show 0.12 for union and 0.24 for intersection [37]. These low scores suggest that simple union or intersection rules do not reproduce the catalog-recorded labels of molecule pairs.

Catalog blend labels differ from the labels of the parent molecules in two ways. In catalog-label loss, a scent word recorded for either parent molecule is absent from the blend's labels. In catalog-label gain, the blend's labels contain a scent word that neither parent has. The results of Sisson et al. show that, on average, 1.2 notes are lost and 0.2 notes are gained per blend. Furthermore, an odds-ratio analysis reveals that certain scents, such as "eggy" or "cabbage", were more likely to be gained, while certain scents like "peach" or "fresh" were more likely to be lost [37]. These are differences between catalog records, not verified perceptual effects. They appear to follow a pattern rather than being random, which motivates testing a model that can learn relationships between the two molecules, such as a Set Transformer.

2.2.2 Evidence That Averaging May Be Sufficient

Not all evidence points to strongly non-additive mixing. Pellegrino et al. [28] had a trained panel rate 432 mixtures using a 51-term list and found that a linear average of the components' ratings predicted mixture profiles about as well as the noise in the panel's own ratings allowed. They argue that apparent emergence often reflects the limits of describing an odor with a few words. Binary catalog labels, as in Sisson et al. [37], instead record catalog-label loss and gain in many blends. Whether a learned pair-aggregation step helps for binary multi-label prediction is therefore an open question, which this study tests rather than assumes.

Other human studies describe more biology-focused ways in which a mixture can be perceived, such as odorants antagonizing one another [9, 13]. This study does not model receptors; the Set Transformer is evaluated only as a statistical aggregator of two learned vectors.

2.3 Machine Learning on Sets and Chemical Mixtures

This section is the core of the chapter. It defines permutation invariance, explains how mean pooling, order-averaged concatenation, and Set Transformer aggregation each combine two molecule vectors, and reviews prior work on mixture modeling.

2.3.1 Permutation Invariance and Deep Sets

A combination of two or more molecules could be viewed as a mathematical set. An accurate depiction of a fragrance blend of these molecules requires that they be presented in such a way that their positioning in that list is arbitrary, not sequential [37, 43, 52]. For instance, assume the combination of molecule A and molecule B produces scent C. It should be of no relevance whether it is molecule A + B or molecule B + A; it should always produce scent C. For that reason, it is important for any mathematical model used to predict the resultant smell from the combination of different molecules to follow this principle, which is called permutation invariance. In plain terms, a function is permutation invariant if swapping its inputs does not change its output.

Zaheer et al. [51] established the math of permutation invariance and introduced Deep Sets, a deep learning framework that uses a neural network designed to process datasets in a non-sequential manner for unordered data. This is referenced as the first of two properties they introduced in their study describing permutation invariance, positing that for a function acting upon a set, it must be indifferent to the order in which each element appears, mathematically formulated as $f : 2^X \to Y$ for any permutation, i.e., for any permutation π:

$$f(\{x_1, \ldots, x_M\}) = f(\{x_{\pi(1)}, \ldots, x_{\pi(M)}\}).$$

Their second result explains how to build such a function: transform each element individually (ϕ), aggregate the results, and then transform the aggregate again (ρ) [51].

However, because each element is transformed before pooling and in isolation, any information about how the elements relate to each other must be inferred indirectly by later layers from the pooled representation, as noted by Lee et al. [24]. For a pair of molecules, this raises the question of whether letting the two vectors affect each other before pooling helps.

2.3.2 Set Transformer (SAB and PMA)

To bridge this gap, Lee et al. propose the use of a Set Transformer, where instead of processing the elements of a set individually, they apply self-attention mechanisms they call SAB (self-attention block) in the processing of the elements, wherein the different elements are able to learn from each other, such that the extracted features to be used for pooling already contain useful information about the elements' relationship with each other. SAB itself is not permutationally invariant so swapping the order of the input gives different outputs. Moreover, they also developed PMA, or Pooling by Multihead Attention, which replaces the second step in the Deep Sets architecture wherein, instead of fixed sum or mean pooling, PMA also employs self-attention mechanisms in the pooling itself, such that it is able to pick out which of the elements bear more weight for the task at hand and gives a permutationally invariant output. Both are candidates for combining the two molecule vectors in this study: SAB lets each vector be updated using the other before pooling, and PMA learns how to weight them. The Set Transformer is therefore treated as a way to model statistical relationships between two learned vectors.

2.3.3 Three Ways to Combine Two Vectors

Three ways of combining the two vectors are compared in this study. Mean pooling averages the two vectors. It is simple and naturally order independent, but it can lose information specific to the pair, because different pairs with the same average produce the same combined representation [41]; this does not mean that it must fail on any particular task. Order-averaged concatenation places the two embeddings side by side in both possible orders, as in the pairwise design of Sisson et al. [37], and averages the two outputs. This adaptation makes the baseline invariant to input order while retaining both embeddings in full. Set Transformer aggregation lets the two vectors affect each other's learned representation before pooling, while remaining independent of order.

The three methods also differ in cost. Self-attention blocks in the Set Transformer scale quadratically with the number of elements in a set [24]; for a set of two molecules this cost is small, but attention blocks still add trainable parameters and computation compared with fixed pooling. A fair comparison therefore reports parameter counts and computation cost alongside accuracy, so that any gain can be weighed against the extra complexity.

2.3.4 Precedents in Mixture Modeling

Zaheer et al. [51] provided the groundwork for permutation invariance in sets, while Lee et al. [24] extended it with self-attention, a general set method that later studies applied to molecular mixtures.

Qin et al. [29] built SolvGNN to predict activity coefficients of binary and ternary chemical mixtures using two levels of graph processing: graph convolutions inside each individual molecule, followed by a molecule-level graph whose nodes are the molecules themselves. They argue that modeling the mixture explicitly at this second level matters in cases where naive pooling falls short [29].

Similarly, Sanchez Medina et al. [35] introduce GH-GNN, which also predicts activity coefficients of chemical mixtures. It uses a single shared molecular encoder whose pooled output is passed to a mixture-level graph. This resembles the structure of the present study, a shared single-molecule encoder followed by a mixture-level step, but applied here to odorant pairs rather than to solvent and solute.

Lastly, Tom et al. [43] proposed POMMix, a more relevant precedent for our topic, as it is in the field of olfactory perception, aiming to predict the similarity of the scent of two mixtures. It employs a module they call CHEMIX, which uses a self-attention mechanism, doing set-pooling to produce a permutation-invariant mixture embedding. POMMix differs from this study in its prediction task: it predicts the similarity between two mixtures, whereas this study predicts the scent labels of a two-molecule blend.

These three studies tackle different prediction problems. Evidence from solvent mixtures in particular does not transfer directly to scent labels. They motivate evaluating learned aggregation for odor blends; they do not establish that it outperforms simple pooling for the binary scent labels used here.

Hierarchical GNNs have also been used for property prediction within a single molecule, moving from atoms to fragments, as in HiGNN and HimGNN [17, 53]. POMMix, SolvGNN and GH-GNN, in contrast, operate across molecules. The present study follows the second kind: a shared single-molecule encoder followed by a pair-aggregation step for the two molecule embeddings.

2.4 Olfactory Datasets and Label Reliability

This section distinguishes the single-molecule catalog labels from the pairwise blend labels, describes what each can and cannot support, and explains why label inconsistency, label noise, and label sparsity matter for evaluation.

2.4.1 Reference Catalogs

The quality of sources for public scent data are not equal. Community platforms such as Fragrantica and Basenotes describe finished commercial perfumes in the words of their users, which makes them unsuitable for learning how individual chemicals relate to scent [7, 46]. Because of these, researchers use catalogs such as GoodScents and Leffingwell for reference, as they record descriptors for individual aroma chemicals and have been used to train state-of-the-art prediction models [23, 37]. The Leffingwell PMP 2001 database was compiled by olfactory experts, usually practicing perfumers, and its free-text descriptions were standardized into 113 descriptors by Sanchez-Lengeling et al.

2.4.2 The Blended-Pair Dataset

Sisson et al. [37] compiled blend labels from the lists of suggested blending partners on GoodScents, which group each partner under an odor heading. Their crawl was able to produce more than 160,000 pairs and, after standardization and filtering, resulted in up to 74 odor labels. Each molecule's page was crawled separately, so a pair can be listed from both molecules' pages. The dataset has no concentration information, and its labels are catalog suggestions and not measurements from an actual sensory panel. In this study they are therefore treated as catalog-recorded blend labels and used as proxy targets.

Note as well that the pair dataset used in this study is not assumed to be identical to the one used by Sisson et al., and that the pair scent labels are separate from the single-molecule labels in GoodScents and Leffingwell, which describe only single molecules.

Furthermore, a scent label may appear from one direction only (e.g., molecule A lists B as a potential blending partner for scent X, but B does not list A). Requiring both molecule pages to list each other for that scent label is one way to reduce reliance on a single recommendation. This study calls this two-page descriptor intersection, which is defined in Chapter 3. The trade-off is that it reduces the number of eligible labels, and it cannot show that a label is perceptually correct.

2.4.3 Human-Rated Datasets

Expert-compiled catalogs such as Leffingwell are assigned by people, but they are used here as training data and cannot serve as an independent check. Independent human-rated data are therefore needed. Human-rated scent datasets exist for single molecules and for mixtures, but to our knowledge none that gives scent descriptors for two-molecule blends is publicly available as of November 2026. Bierling et al. [6] provide yes-or-no ratings of 16 scent words for 74 single odors from about 1,200 laypeople. Pellegrino et al. [28] collected trained-panel ratings of mixtures with a 51-term list.

2.4.4 Absence of a Standard Vocabulary

No standard set of odor descriptors exists. Catalogs and studies use vocabularies of different sizes, and many words overlap in meaning or differ only in spelling, such as “acid” and “acidic” [37]. Sajan et al. [34] grouped raw odor words from the Pyrfume repository into 16 broader scent families, but this study does not use those families because their words are not directly equivalent to the descriptors in the present data. The models therefore predict only the standardized descriptors defined for this dataset. How the vocabulary is standardized and filtered, and how unlisted descriptors are treated, is described in Chapter 4.

2.4.5 Noisy and Imbalanced Labels in Multi-Label Learning

Inconsistent labels act as noise in both the training targets and the test targets. Noise in the training targets can change what a model learns, because deep networks can memorize noisy labels [38]. Noise in the test targets can change which model appears best. Noise is harder to handle with several labels per example, because errors can be correlated across labels; Li et al. [25] model this by estimating a noise transition matrix that uses label correlations. Moreover, missing scent labels is an additional problem because unlisted scent labels may not necessarily be true absence and were just omitted by the catalog. For this reason, this study measures label consistency, evaluates on the most consistent labels, and tests whether the comparison between aggregation methods changes as label reliability decreases.

Multi-label scent data are also sparse: most descriptors are absent for most molecules, and some descriptors are far more common than others. Reviews of imbalanced multi-label classification describe imbalance within labels, among labels, and among label sets [42], and work on graph neural

networks reports that the benefit of balancing techniques depends on the dataset and problem type [2]. Because simple accuracy-style scores can look good on sparse labels, evaluation of such data needs metrics that are not dominated by true negatives [30].

2.5 Out-of-Distribution Splitting Strategies

This section provides a review of the standard data-splitting strategy of scaffold splitting, and an explanation of how its application could fall short in sparse chemical spaces. This section then grounds and discusses the clustering-based alternative used in this study.

A common method to partition data for training, testing, and validation in chemoinformatics and molecular machine learning is scaffold splitting [15, 39]. In this method, molecules are represented by graphs of their structure, and the core framework of these graphs is referred to as their "scaffold" (Bemis–Murcko scaffolds) [48]. Molecules that share the same scaffold are grouped together, and these groups are then distributed among the training, testing, and validation sets to ensure that the test set contains structurally different molecules from the training set, which is a basic form of out-of-distribution (OOD) evaluation, which as opposed to random splitting, simply distributes molecules randomly across training, validation, and test sets [12, 15, 37, 39].

The caveat with this method is that, when the dataset is sparse, it collapses toward random splitting. For instance, similarly structured molecules could get fragmented across both the train and test splits, leading to data leakage and inflated test scores as scaffolds in standard benchmarks could be populated by only one or two molecules each, undermining the grouping effect the method depends on [12]. Another problem demonstrated by scaffold splitting is that even molecules that are near-identical (as computed by Tanimoto similarity) [15, 39], even differing by as little as a single atom, can, under this method, end up in opposite splits, as observed by Guo et al. and Steshin et al. [15, 39], who argue that this problem causes models to "memorize" rather than generalize from the data. Because of these problems, scaffold splitting can misleadingly produce scores that are overly optimistic.

Shimakawa et al. [36] propose doing topological pre-clustering by projecting 2048-bit ECFP4 fingerprints onto a 2D map using 2D Uniform Manifold Approximation and Projection (UMAP), then grouping them together using spectral k-means clustering. More generally, leakage-reduced splitting can be formulated as an optimization problem over the data [22], and cluster-based splits

have been reported to be among the most challenging for both classical and graph-based models [12].

In plain terms, one molecule can appear in many blend pairs, so splitting pairs at random can place the same molecule in both training and testing. Methods designed for single molecules do not address this, because an individual molecule can be a component of varying other molecules in different blends, meaning a molecule seen during training could reappear as a component in a different, unseen test blend, leaking information about that molecule across the split and inflating the performance metrics. Therefore, Sisson et al. [37] propose "graph carving", using a meta-graph to describe the pairwise blend dataset, where each node represents an individual molecular graph and each edge represents a tested blend combination. Edges connecting a molecule from one partition to a molecule in another are discarded, so that no component is shared between splits. In their original implementation, this partitioning was done by randomly assigning molecules to each side and repeating until every label had at least one training and one test example, since a true scaffold-based split was not compatible with their sparse label distribution.

This study applies graph-carving protocol on top of a fingerprint-clustering partition (in the style of the UMAP and k-means approach discussed above), rather than random or scaffold-based partitioning. Doing this creates a component-disjoint split that reduces component and structural overlap between training and testing; it does not guarantee that every test molecule is entirely unlike every training molecule.

2.6 Evaluating and Comparing Models Under Label Sparsity

Sparse multi-label targets make some common scores misleading. Precision, recall, and F1 ignore true negatives and so expose a model that predicts no labels, whereas accuracy-style scores such as Hamming loss can look strong for such a model [30]. F1 can be averaged per descriptor (macro), which gives scent labels that are rarely found the same weight as common ones, or pooled across all decisions (micro), which favors common scent labels [30]. Average precision (area under the precision–recall curve) summarizes how well a model ranks true scent labels above false ones across all cutoffs. The cutoff used to turn a probability into a present or absent prediction is also important. A study comparing graph-based and traditional models for multi-label odor prediction found that choosing the right threshold was essential [45]. To ensure fair results, this threshold should be

selected without using the test set.

Comparisons between models should also report how large a difference is. Bootstrap confidence intervals do this by showing the difference and its uncertainty together [5]. Recent work warns against naive paired t-tests when resampled test sets are used [30]. Together with the leakage problem above, this means that a fair comparison of aggregation methods needs matched encoders and training conditions, sparsity-aware metrics, validation-only threshold selection, and paired uncertainty estimates over the evaluated blends.

2.7 Synthesis and Research Gap

2.7.1 Synthesis of Literature

Current models in computational olfaction can successfully map single-molecule structures to their appropriate sensory descriptors [23]. However, because these compounds are evaluated in isolation, they do not address how two molecules should be combined, and catalog blend labels differ from the labels of the parent molecules in many blends [37]. Human-panel evidence on mixtures is mixed: continuous ratings are well predicted by a linear average of the components [28], while binary catalog labels record catalog-label loss and gain in many blends [37].

From an architectural standpoint, order-independent set processing is a natural requirement for combining unordered components [51, 24]. Averaging can make different inputs look identical, and a related loss of information has been described for averaging-based readouts within a molecule [41]. For a pair, this is a possible limitation of mean pooling.

Furthermore, standard scaffold splits can behave like a random split and can cause data leakage [12]. Clustering molecules and carving the blend meta-graph along cluster boundaries reduces component and structural overlap between training and testing [36, 37]. Finally, the blend labels in public data come from catalogs rather than sensory panels, and their reliability has rarely been measured in prior blend studies.

In summary: (1) a molecular graph encoder produces one vector per molecule; (2) the unresolved computing problem is how to combine the two vectors without making the prediction depend on input order; and (3) any such comparison must account for component leakage and uncertain labels.

2.7.2 Statement of the Research Gap

To our knowledge, existing work does not yet establish whether an attention-based set aggregator provides a reliable advantage over simpler alternatives when two molecular graph embeddings are combined for sparse, catalog-derived multi-label prediction. A fair answer requires the encoder and training conditions to be controlled, input order to be handled explicitly, repeated molecular components to be separated across partitions, and uncertain labels to be considered during evaluation. This study addresses that controlled evaluation gap.

2.7.3 Positioning of the Present Study

To address this gap, this study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for combining the two molecule embeddings produced by one shared, frozen graph attention encoder, with a separately trained prediction head for each method. The comparison is carried out on a component-disjoint, fingerprint-clustered and graph-carved partition of the data. The two-page descriptor intersection screen and the controlled label-corruption tests show how far the conclusions depend on the labels. A simpler baseline that matches or outperforms the Set Transformer would be a valid and informative result.

Chapter 3

Theoretical and Conceptual Framework

3.1 Theoretical Framework

This chapter follows the computation from the molecular input to the final evaluation. It first explains how a molecule is represented as a graph of atoms and bonds. It then describes how message passing and graph attention produce one embedding for each molecule. The next part explains why combining two embeddings is a separate design problem: a blend is an unordered pair, so the prediction should not depend on which molecule is listed first. The three aggregation methods are then described and compared. The chapter ends by explaining why sparse, catalog-derived labels affect both training and evaluation, and how reciprocal agreement, cross-catalog agreement, and controlled label corruption are used to study that problem. The discussion of averaging describes a possible information loss, not an observed result; the relative performance of the methods remains an empirical question.

3.1.1 Representing Molecules as Graphs

Each molecule is represented as an undirected, attributed graph. Atoms are the nodes, bonds are the edges, and the features attached to them become the inputs to the graph encoder [49]. This representation keeps the bond connectivity of the molecule instead of treating the structure as an

ordinary sequence of characters. SMILES strings are used to store and parse the structures, but different valid atom-numbering or starting-point conventions should describe the same molecular graph [27, 46].

Mathematical Formalism

Each molecule is defined as an undirected, attributed graph $G = (V, E)$, where V is the set of non-hydrogen atoms and E is the set of covalent bonds. In the representation used here, hydrogen information is included in the features of the corresponding heavy atoms rather than represented as separate nodes [4].

The connectivity can be written as an adjacency matrix A, where $A_{ij} = 1$ when atoms i and j are connected by a bond and $A_{ij} = 0$ otherwise [53]. The implementation stores this connectivity in a sparse form (Section 3.3.2). Each node and edge also has a feature vector:

- The node vector $\mathbf{x}_i \in \mathbb{R}^{d_x}$ records properties such as element, charge, valence, hybridization, aromaticity, hydrogen count, ring information, and chirality.
- The edge vector $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$ records bond properties such as bond type, conjugation, ring membership, and stereochemical configuration.

The complete node and edge feature tables are provided in Appendix A. RDKit supplies the parsed molecular structures and their features. The graph representation is not tied to a particular atom numbering: relabeling the nodes changes their order in the data structure, but not the bonds or the molecule they describe. This property is important because the message-passing and pooling operations should produce the same molecule representation when the atoms are listed in a different order. In this study, $d_x = 46$ and $d_e = 10$, with the components listed in Appendix A.

3.1.2 Graph Neural Networks and Message Passing

Most neural networks receive inputs in a fixed arrangement. A molecular graph does not have that form: molecules can contain different numbers of atoms, and their important relationships are represented by bonds. A Graph Neural Network (GNN) works with this structure directly. In the graph used here, a *node* represents an atom and its feature vector, while an *edge* represents a bond and its feature vector. The *neighborhood* $\mathcal{N}(i)$ of node v_i is the set of atoms directly connected to it by an edge [53, 4, 45].

The main operation in a GNN is *message passing*. During one layer, each node receives information from its neighborhood, combines those messages, and updates its own representation. For node v_i at layer t, one general form of this operation is:

$$\begin{aligned} \mathbf{m}_i^{(t)} &= \bigoplus_{j \in \mathcal{N}(i)} M_t\big(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}, \mathbf{e}_{ij}\big), \\ \mathbf{h}_i^{(t+1)} &= U_t\big(\mathbf{h}_i^{(t)}, \mathbf{m}_i^{(t)}\big). \end{aligned} \tag{3.1}$$

Here, $\mathbf{h}_i^{(t)}$ is the current representation of node i, $\mathbf{e}_{ij}$ describes the bond to neighbor j, and M_t computes the message sent along that bond. The symbol $\bigoplus$ denotes the operation that combines all messages received by node i, while U_t produces its new representation. In ordinary terms, the node looks at itself, looks at its bonded neighbors, and then records an updated summary. Since the messages are combined as a group rather than in a prescribed order, changing the order in which neighboring atoms are stored does not change the information being combined. Repeating the layers allows information to travel from an atom to nearby atoms and then to larger parts of the molecule [17, 53].

After the final message-passing layer, a *graph readout* combines the node representations into one vector for the whole molecule. The readout specified for the final protocol is applied consistently to both molecules; when mean pooling is used, the node vectors are averaged before being passed to the blend-level models. This node-to-molecule step is common in molecular GNNs, although some recent models also learn representations for larger substructures [17, 45].

The GNN used here is a Graph Attention Network (GAT). Rather than assigning the same fixed weight to every neighboring node, a GAT learns weights from the node and bond features, allowing the encoder to give more influence to some local connections than to others. These weights are model parameters used to form a representation for descriptor prediction; they do not establish a receptor mechanism or a physical interaction between molecules [4, 45]. The specific attention calculation is described in the next subsection.

3.1.3 Graph Attention Encoder

The shared graph encoder maps each molecular graph to a fixed-length vector that can be used by any of the three blend-level aggregation methods. It learns representations of local atom and bond context that are useful for the descriptor prediction task. The same encoder is applied to the two molecules in a blend, so the resulting vectors are produced under the same learned rules [23, 49].

Let $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ denote the 46-dimensional initial feature vector of node v_i, and let $\mathbf{e}_{ij}$ denote the 10-dimensional edge feature vector. The approved encoder in Chapter 4 uses three graph-attention layers and produces a 256-dimensional molecule embedding. A learnable matrix projects the node states into the hidden space. To score how much node v_i should attend to neighbor v_j, a small learnable scoring function uses the projected states of both nodes and the edge features [49, 53]:

$$\tilde{e}_{ij} = \mathrm{LeakyReLU}\Big(\mathbf{a}^T\big[\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j \,\|\, \mathbf{W}_e\mathbf{e}_{ij}\big]\Big), \tag{3.2}$$

where $\|$ denotes concatenation. Including $\mathbf{e}_{ij}$ lets the score use information about the bond connecting the two nodes. The activation and its parameters are part of the approved encoder specification [49].

Atoms have different numbers of neighbors, so the raw scores are normalized across the neighborhood $\mathcal{N}(i)$ of atom v_i with a softmax, so that the weights for each atom sum to one [49, 4]:

$$\alpha_{ij} = \mathrm{Softmax}_j(\tilde{e}_{ij}) = \frac{\exp(\tilde{e}_{ij})}{\sum_{k\in\mathcal{N}(i)}\exp(\tilde{e}_{ik})}. \tag{3.3}$$

The normalized weights α_{ij} scale the projected neighbor features, and the selected node-update activation produces the updated representation of node v_i [49]:

$$\mathbf{h}_i^{(l+1)} = \sigma_{\mathrm{upd}}\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}\mathbf{W}\mathbf{h}_j^{(l)}\right). \tag{3.4}$$

Here σ_{upd} denotes the node-update activation selected in the final encoder specification.

The encoder repeats this update across three layers. The first two use four attention heads, and the last produces the final 256-dimensional node states. A learned global-attention readout then combines the final node states into one molecule vector $\mathbf{z}$ passed to the blend-level model. Thus, the node states retain local and progressively broader graph information, while the readout makes that information available as one vector for the molecule.

The encoder is pretrained once on the training-partition single molecules and then frozen for the primary comparison. The three aggregation variants therefore receive embeddings produced by the same encoder weights and use matched training schedules, optimization settings, and budgets.

The attention coefficients are learned computational weights used to form the representation. They indicate which neighboring features the model emphasizes for its prediction, but they do not demonstrate chemical importance, receptor activity, or a physical interaction between molecules.

3.1.4 Combining an Unordered Pair

The blend model receives two molecule embeddings, $\mathbf{z}_A$ and $\mathbf{z}_B$. Because the pair has no meaningful first or second member, the model should return the same prediction for (A, B) and (B, A). In other words, changing the order in which the two molecules are supplied must not change the predicted descriptor vector [37, 51].

The three aggregation methods satisfy this requirement in different ways. Their pair representations and main trade-offs are summarized below.

Table 3.1: Aggregation methods for an unordered pair of molecule embeddings.

Method	**Pair representation**	**How order is handled**	**Main trade-off**
Mean pooling	Average the two embeddings	Invariant by construction	Can discard pair-specific distinctions
Order-averaged concatenation	Evaluate both concatenation orders and average their predictions	Explicitly symmetrized	Requires two ordered evaluations
Set Transformer aggregation	Let the two embeddings exchange learned information before pooling	Invariant set processing	Adds parameters and computation

Mean Pooling

Mean pooling is the simplest order-independent baseline. It represents the pair by the element-wise average

$$\mathbf{z}_{\text{mean}} = \frac{\mathbf{z}_A + \mathbf{z}_B}{2}. \tag{3.5}$$

Swapping the two inputs gives the same vector automatically, so this method is invariant by construction. Its limitation is that different pairs can share the same average. For example, the one-dimensional pairs $(2, 8)$ and $(4, 6)$ both produce 5. This possible loss of pair-specific distinction motivates the comparison with the other two methods; it is not a claim about the observed results [51, 41].

Order-Averaged Concatenation

Concatenation keeps the two embeddings side by side, but one ordered concatenation depends on which molecule is listed first. The adapted baseline therefore evaluates both $[\mathbf{z}_A \,|\, \mathbf{z}_B]$ and $[\mathbf{z}_B \,|\, \mathbf{z}_A]$,

then averages the two resulting model outputs. This explicit symmetrization makes the final prediction the same for both input orders while retaining more pair-specific information than a simple average. It requires two ordered evaluations for each pair.

The two logits are averaged before a single sigmoid is applied. This is an adapted baseline based on the pairwise concatenation design of Sisson et al., not a reproduction of their complete model [37].

Set Transformers and Attention-Based Pooling

The Set Transformer receives the two embeddings as a set $X = \{\mathbf{z}_A, \mathbf{z}_B\}$. Its attention blocks allow the embeddings to exchange learned information before a pooling step produces one blend representation [24, 43]. The Self-Attention Block (SAB) processes the two members without assigning them a permanent first or second position, so the resulting set representation is unchanged when the inputs are swapped. The method can represent statistical relationships between the learned embeddings that mean pooling may discard, but it adds parameters and computation. The approved configuration uses two SAB blocks with four attention heads and a 512-unit row-wise feedforward layer.

The second block is Pooling by Multihead Attention (PMA):

$$\mathrm{PMA}_k(X) = \mathrm{MAB}\big(S, \mathrm{rFF}(X)\big). \tag{3.6}$$

This process begins by passing the molecular embeddings $\mathbf{X}$ through a row-wise feedforward transformation. The PMA block uses learned seed vectors S to attend to these transformed embeddings and compress them into a fixed-size blend representation [24, 37]. In this study, PMA uses one learned seed vector, and its attention weights are interpreted as computational parameters for combining representations. They model statistical relationships between embeddings for descriptor prediction, not physical molecular interactions or receptor mechanisms. Whether the additional capacity improves prediction is tested empirically under the same training and evaluation conditions as the baselines.

3.1.5 Representation Trade-offs

The three methods make different compromises when two molecule embeddings are reduced to one blend representation. Mean pooling is inexpensive and order independent, but different pairs

can have the same average, so some pair-specific distinctions may be lost [51, 41]. It remains a credible baseline because its simplicity can be sufficient when the averaged embedding retains the information needed for the descriptor task. The order-averaged concatenation baseline retains the two embeddings side by side and restores order invariance by evaluating both input orders, at the cost of a second ordered evaluation. Set Transformer aggregation has additional learned capacity and computation, and uses that capacity to model statistical relationships between the embeddings before pooling. These are representational trade-offs, not assumptions about the chemistry of a blend.

Why Averaging Can Lose Information

Averaging two molecule embeddings can make structurally different pairs indistinguishable to the model. In one dimension, the pairs $(2, 8)$ and $(5, 5)$ both average to 5, so the averaged representation cannot distinguish those pairs. This example demonstrates a possible information loss; it does not show that such collisions are common or dominant in the present dataset. The actual usefulness of mean pooling remains an empirical question. The order-averaged concatenation baseline retains the two embeddings side by side while correcting for the two possible input orders, and the Set Transformer provides another order-invariant way to combine them by learning statistical relationships before pooling. A full derivation of the symmetry argument is given in Appendix A.1.

3.1.6 Sparse Labels and Label Reliability

The output is a sparse multi-label vector: most blend–descriptor cells are absent, while a smaller number are recorded as present. This imbalance affects both parts of the experiment. During training, the many easy negative cells can dominate the loss. During evaluation, a model that predicts mostly negatives can look favorable on an aggregate score even when it misses useful positive descriptors. The study therefore uses metrics suited to sparse multi-label outputs and applies a reliability mask when only the more consistent cells are being evaluated. The exact mask construction and training conditions are specified in Chapter 4.

Because the blend labels used in this study are compiled from catalog recommendations rather than a controlled sensory panel, their reliability must be considered before model comparisons can be interpreted. The reliability checks describe the strength and consistency of the catalog evidence; they do not establish perceptual truth.

Reciprocal agreement is a within-catalog check. In the GoodScents catalog, molecule A's page may list molecule B as a blending partner under a given scent word, but molecule B's page may not list molecule A under the same word. When both pages agree, the label is *reciprocal*; when only one page makes the claim, the evidence is weaker. Reciprocal evidence therefore strengthens confidence in a recorded positive label, but it does not prove that the descriptor is perceptually correct.

Cross-catalog agreement is measured using Cohen's kappa (κ), which quantifies how consistently two independently maintained catalogs (GoodScents and Leffingwell) assign the same descriptor to the same molecule, correcting for the agreement expected by chance alone. A commonly used benchmark scale classifies κ values between 0.21 and 0.40 as "fair" agreement, 0.41 to 0.60 as "moderate," and above 0.60 as "substantial" agreement [44]. These categories are used as operational descriptions of descriptor-level consistency, not as universal or objective judgments of label correctness. The thresholds and the catalogue records used to compute them are specified in Chapter 4.

The target representation follows a closed-world assumption for the purpose of the supervised task: an unlisted descriptor is encoded as a negative target. This is a modeling convention, not a claim that the odor is truly absent; catalogs can omit descriptors for reasons unrelated to sensory absence.

Reliability is applied at the blend–descriptor cell level. An eligibility-mask comparison must include eligible negative cells as well as reliable positive cells. Otherwise, a test set containing only recorded positives could not penalize false-positive predictions, making measures such as F1 or PR-AUC misleading. Cells whose evidence is ambiguous are masked from the relevant loss or metric rather than causing the entire blend to be discarded. Weaker evidence does not prove that a catalog label is wrong.

Even after reliability screening, some label noise may remain. The controlled label-corruption experiment described in Chapter 4 tests how the aggregation methods respond as training-label reliability is reduced. A smaller performance decline under the same corruption condition would indicate greater robustness to that tested form of noise, but it would not establish that the labels are perceptually accurate [38].

3.2 Conceptual Framework

The conceptual framework of this study operationalizes the theoretical formulations into a systematic computational pipeline structured around the Input-Process-Output (IPO) model. Figure 3.1 outlines the structural interaction between input data representations, the two-level hierarchical deep learning architecture, and the downstream analytical and practical deliverables.

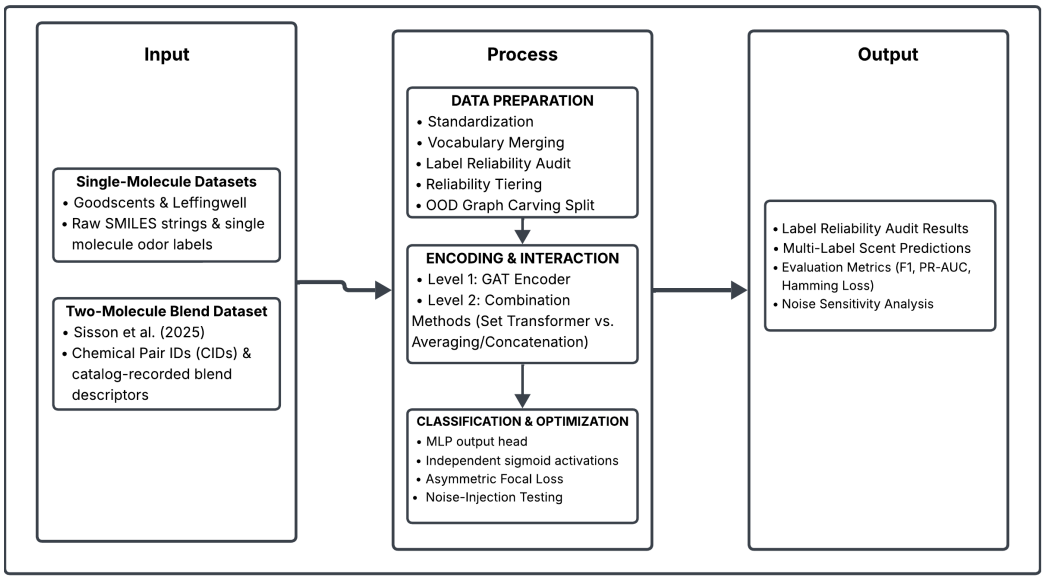


Figure 3.1: Input-Process-Output (IPO) Model.

3.2.1 Input Phase

The computational data pipeline starts with the acquisition, curation, and gathering of data from olfactory databases. The raw SMILES strings are extracted from the GoodScents and Leffingwell repositories accessed via the Pyrfume Library in Python [7]. These raw chemical blueprints are then paired with catalog-recorded odor tags as proxy targets for pretraining. The binary mixture profiles are retrieved from the Sisson et al. (2025) benchmark dataset [37]. The retrieved benchmark dataset provides standardized chemical pair identifiers (CIDs) and catalog-recorded descriptors that will be used for multi-label prediction of two-molecule blends.

3.2.2 Process Phase 1: Data Preparation and Graph Carving

To raw chemical input will first pass through a standardization pipeline. Salts and other non-fragrance components are removed using RDKit's SaltRemover module, leaving the main molecule. Compounds containing elements other than C, H, O, N, S, F, Cl, Br, and I are dropped, since these fall outside the range typical of aroma chemicals [27]. The molecular weight should be between 20 and 600 g/mol. Below this range is considered irrelevant, while above the range is considered too heavy to be perceived as a scent at all [23]. The surviving SMILES strings are canonicalized using RDKit. The scent descriptors text from the source databases is standardized by merging redundant terms and eliminating descriptors that appear rarely to learn from [8]. The blend labels are then audited for reliability before any model is trained. Reciprocal agreement is measured for each blend pair, and cross-catalog agreement between GoodScents and Leffingwell is measured on the parent molecules using Cohen's kappa. The resulting evidence is used to construct the cell-level reliability conditions described in Chapter 4. These conditions describe stronger or weaker catalog evidence; they do not prove that a label is true or false. Unlisted descriptors are treated as negative targets under the study's closed-world assumption, while ambiguous cells are masked rather than used to remove an entire blend. To test how well the network handles unfamiliar compounds, molecules are grouped into structural families by the approved clustering procedure. The Graph-Carving algorithm then divides the mixture meta-graph along family clusters. The resulting component-disjoint procedure reduces overlap of molecular components between partitions; it does not guarantee that held-out molecules are structurally unrelated to training molecules [12].

3.2.3 Process Phase 2: Encoding and Pair Representation

The second phase converts each molecular graph into a fixed-length vector. Atoms and bonds are processed as graph nodes and edges, and the shared GAT encoder updates the node representations before applying global-attention readout [49]. The same frozen encoder is used for both molecules and all three aggregation variants. The dimensions and layer settings follow the fixed specification in Chapter 4.

The two molecule embeddings are then combined into a single blend representation using one of three aggregation methods: mean pooling, order-averaged concatenation following the adapted pairwise baseline of Sisson et al. [37], or Set Transformer aggregation. These methods are compared

under matched training schedules. The Set Transformer is treated as a learned way to combine the two representations; the study does not claim that its attention weights describe physical or receptor-level interactions.

3.2.4 Process Phase 3: Classification and Optimization

Once the aggregation module yields the blend representation, it is translated into multi-label descriptor predictions. The Multi-Layer Perceptron (MLP) head maps the 256-dimensional blend representation through 128 hidden units and ends with L individual outputs to match the target scent classes [37]. Because scent profiles are inherently multi-label, the classification layer utilizes independent sigmoid activation functions [33]. These activations compute individual probability scores for the joint presence or absence of each unique target scent descriptor. Because the encoder is frozen for the primary comparison, prediction errors update only the aggregation and output modules. The loss uses the fixed imbalance parameters stated in Chapter 4 [18]. Each combination method is trained under the reliability conditions specified in Chapter 4, including a condition that uses eligible high-reliability cells and their eligible negative cells. The noise-sensitivity test applies the declared controlled label corruption to the same matched conditions. Olfactory datasets are highly sparse, meaning a single scent profile contains only a few active descriptors alongside dozens of inactive ones (zeros). Under standard training rules, the model could appear accurate simply by predicting "no" for most labels. The asymmetric loss addresses this class imbalance by reducing the influence of obvious negatives and forcing the network's mathematical focus onto the rare, active scent notes that are difficult to predict [18].

3.2.5 Output Phase

The final phase outputs a multi-label binary vector of predictions across all L target scent descriptors which is based on the probability given to each descriptor. A validation module evaluates the model's performance on these predictions under the declared reliability masks. It calculates metrics such as macro- and micro-averaged F1-scores, Hamming loss, and the Precision-Recall Area Under the Curve (PR-AUC) [3, 42, 45]. The reliability-aware comparison includes eligible negative cells so that false-positive predictions remain evaluable; ambiguous cells are excluded by the mask. The exact partitions, masks, and statistical comparison procedure are specified in Chapter 4.

The study additionally reports the results of the label reliability audit, including reciprocal agreement rates and cross-catalog agreement by descriptor.

3.3 Technical Background

Mapping molecular structure onto multi-label olfactory predictions requires stitching together a few different layers of infrastructure: cheminformatics standards to make sure the molecules are valid, a graph learning framework to process them, and a loss function suited to how sparse the labels are. This section walks through each piece—the parsing standards, the sparse graph representation, and the loss objective—and explains why each choice was necessary rather than incidental.

3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization

Molecular structures in this study are represented as Simplified Molecular Input Line Entry System (SMILES) strings, a compact text-based notation, which are canonicalized and parsed into undirected, attributed molecular graphs using the open-source RDKit library in Python [27]. SMILES strings are convenient because they are compact and suitable for storage. This compactness comes at a cost, however: they are structurally sensitive, prone to syntax errors, and do not encode explicit 3D geometry [46]. Before any structure reaches the model, every string passes through RDKit's sanitization pipeline, which runs four checks in sequence:

- Kekulization checks that the alternating single and double bonds within conjugated or aromatic rings are represented in a chemically consistent manner, resolving ambiguous resonance structures.
- Aromaticity Perception evaluates the coordinate and topological structure of each atom to identify aromatic rings, dynamically reassigning aromatic bond attributes to relevant edges.
- Valence Checking checks every atom, specifically heavy atoms–carbon, nitrogen, oxygen, sulfur, and halogen to ensure it does not exceed its physical valency limits. This rejects syntactically corrupted SMILES strings or physically impossible hypervalent structures.
- Ring Perception, which identifies and indexes ring systems, information the later convolutions depend on.

Enforcing these rules ensures that what the network receives is already known to be chemically valid, rather than requiring the model to infer the rules of organic valency implicitly from training data [4, 53]. The sanitization pipeline acts as an explicit structural filter ahead of the model, so that the Level 1 Graph Attention Network (GAT) operates only on topologically valid graphs that correspond to real molecular structures [23, 49].

3.3.2 Deep Learning Stack (PyTorch / PyTorch Geometric): Sparse Graph Convolutions

After sanitization, each molecule is formatted as an undirected graph $G = (V, E)$, where atoms are nodes (V) and covalent bonds are edges (E). A dense $N \times N$ adjacency matrix could represent this connectivity, but for molecules, where most atoms bond to only a few neighbors, this wastes memory, scaling at $\mathcal{O}(N^2)$ for structures that are inherently sparse. The architecture instead runs on PyTorch Geometric, which handles molecular graphs as sparse attributed objects rather than dense arrays [11].

Topology is stored in Coordinate (COO) format: a sparse two-row index matrix C of size $2 \times 2E$, where the first row lists source node indices and the second lists receiver node indices [4]. Since chemical bonds are physical and undirected, every source node is also a receiver, so $2E$ entries are needed rather than E — each bond is represented in both directions so that message-passing can update both of its atoms. Node states are updated by running the GAT computation described in Section 3.1.3 over this sparse index matrix, rather than over a dense adjacency matrix, at each layer l.

3.3.3 Multi-Label Mathematical Formulation: Independent Sigmoids and Sparsity Mitigation

A molecule or blend can carry several scent descriptors at once (for example, both fruity and green), rather than belonging to a single exclusive category. The descriptors can therefore co-occur, and predicting one descriptor should not force all other descriptors to be absent. The final layer skips a mutually exclusive softmax and instead maps the approved blend embedding through an MLP to L independent sigmoid outputs, one per target descriptor [33]. Each output is a probability $\hat{y}_{ik}$

between 0 and 1 for example i and descriptor k:

$$\hat{y}_{ik} = \sigma(s_{ik}) = \frac{1}{1 + e^{-s_{ik}}}. \tag{3.7}$$

Here s_{ik} is the output score before the sigmoid. The target y_{ik} is binary for each descriptor, so the model makes one present/absent decision per descriptor rather than one choice from a mutually exclusive list.

A multi-label dataset is sparse when the number of positive cells is small relative to the total number of example–descriptor cells. Let N be the number of examples and L the target-vocabulary size. The label density can be described without fixing a provisional value as $\rho = (NL)^{-1} \sum_{i=1}^{N} \sum_{k=1}^{L} y_{ik}$. A small ρ means that most cells are negative. The many easy negatives can then dominate a plain loss, allowing a model that predicts mostly zeros to appear successful while missing the positive descriptors that matter for the comparison.

A standard multi-label setup would optimize independent Binary Cross-Entropy terms across the L descriptors:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{NL} \sum_{i=1}^{N} \sum_{k=1}^{L} \Big[y_{ik} \log(\hat{y}_{ik}) + (1 - y_{ik}) \log(1 - \hat{y}_{ik}) \Big]. \tag{3.8}$$

The planned loss is an Asymmetric Focal Loss. It separates the focusing of positive and negative terms through exponents γ_p and γ_n and uses a class-frequency weight α_k [18]:

$$\mathcal{L}_{\mathrm{FL}} = -\frac{1}{NL} \sum_{i=1}^{N} \sum_{k=1}^{L} \Big[y_{ik} \alpha_k (1 - \hat{y}_{ik})^{\gamma_p} \log(\hat{y}_{ik}) + (1 - y_{ik})(1 - \alpha_k) \hat{y}_{ik}^{\gamma_n} \log(1 - \hat{y}_{ik}) \Big]. \tag{3.9}$$

The exponents and class weights determine how strongly easy negative terms are down-weighted relative to rarer positive terms. Chapter 4 fixes $\gamma_p = 0$ and $\gamma_n = 2$ for the comparison. The purpose of the loss is to reduce the effect of the imbalance; it does not remove the need for reliability masks or appropriate evaluation metrics.

3.3.4 Evaluation Metrics for Sparse Multi-Label Outputs

Macro F1 is the primary metric for the comparison. It computes F1 separately for each evaluated descriptor and then averages those values, so frequent descriptors do not determine the result by themselves. This is useful when the study compares performance across a vocabulary whose descriptor frequencies may differ.

Micro F1 is reported as a supporting metric by pooling true positives, false positives, and false negatives across descriptors before calculating one F1 value. PR-AUC is also supporting: it measures how well the model ranks positive cells across decision thresholds, which is informative when positives are rare. Hamming loss reports the fraction of evaluated cells classified incorrectly, so it gives a complementary view of overall cell-level errors. The metric definitions and masking rules are specified in Chapter 4.

Hamming loss must be interpreted against an all-negative reference because a sparse label matrix can make an always-negative predictor appear strong. Let M_{ik} be the evaluation mask and define $\hat{y}_{ik}^{(0)} = 0$ for every eligible cell. The corresponding reference is

$$\mathrm{HL}_{\mathrm{neg}} = \frac{\sum_{i=1}^{N} \sum_{k=1}^{L} M_{ik} y_{ik}}{\sum_{i=1}^{N} \sum_{k=1}^{L} M_{ik}}. \tag{3.10}$$

This is the error made by predicting negative for every evaluated cell. A model's Hamming loss should therefore be reported with this reference rather than interpreted in isolation. The reference is computed separately for each declared evaluation mask.

Chapter 4

Methodology

This chapter describes the preparation of the catalog-derived datasets, the construction of reliability masks, the formation of component-disjoint data partitions, and the matched training and evaluation of three aggregation methods. It also defines the robustness conditions, metrics, and paired statistical comparisons used to answer the research questions. Because this is a proposal, the procedures are written in the future tense. Quantities that depend on running the finalized pipeline are identified as values that will be reported, rather than presented as results in advance.

4.1 Research Design

This study will use a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Fragrance-blend prediction is the application. The computer-science investigation concerns how an unordered pair of graph embeddings should be aggregated, how uncertain labels should be handled, and how models should be evaluated when the two molecules of a pair may also appear in other pairs.

The procedures in this chapter operationalize the research questions stated in Chapter 1. Experiments A and B provide the matched aggregation comparison for RQ1, the blend-type analysis addresses RQ2, and controlled label corruption addresses RQ3.

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. Label-reliability and label-corruption conditions are robustness factors, and the

catalog-label pattern of a blend is used only for a secondary subgroup analysis. Everything else is a matched control, so that a difference between aggregation methods is not caused by a difference in data, training budget, or luck of the seed.

Table 4.1: Varied factors and matched controls.

Category	**Treatment**
Aggregation method	Primary varied factor (three levels)
Label-reliability and corruption conditions	Robustness factors
Catalog-label pattern	Secondary subgroup analysis
Encoder design and initialization policy	Matched control
Train/validation/test partitions	Fixed across variants
Loss, optimizer, stopping rule, training budget	Matched controls
Random seeds and corrupted datasets	Paired across variants

Figure 4.1 gives a compact view of the complete computational pipeline. The larger flowchart can be retained for the appendix or presentation slides with its audit and implementation details; this figure keeps the comparison that the panel needs to follow in the main chapter.

The dependent variables are macro F1 (the primary metric, declared before any experiment is run), micro F1, PR-AUC, and Hamming loss, defined in Section 4.7.

4.2 Data Acquisition and Preparation

This section describes the data sources, how they are joined, and how the model inputs (molecular structures) and the model targets (odor descriptor labels) are prepared. The two are prepared separately: structures become graphs, and descriptors become a fixed list of labels.

4.2.1 Data Sources and Provenance

The study will use three sources. The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries. Each record lists a molecule, its structure, and the scent descriptors assigned to it (for example, fruity or sweet). The Leffingwell descriptors are described as assigned by olfactory experts, usually practicing perfumers [23]. The blend source is the two-molecule dataset of Sisson

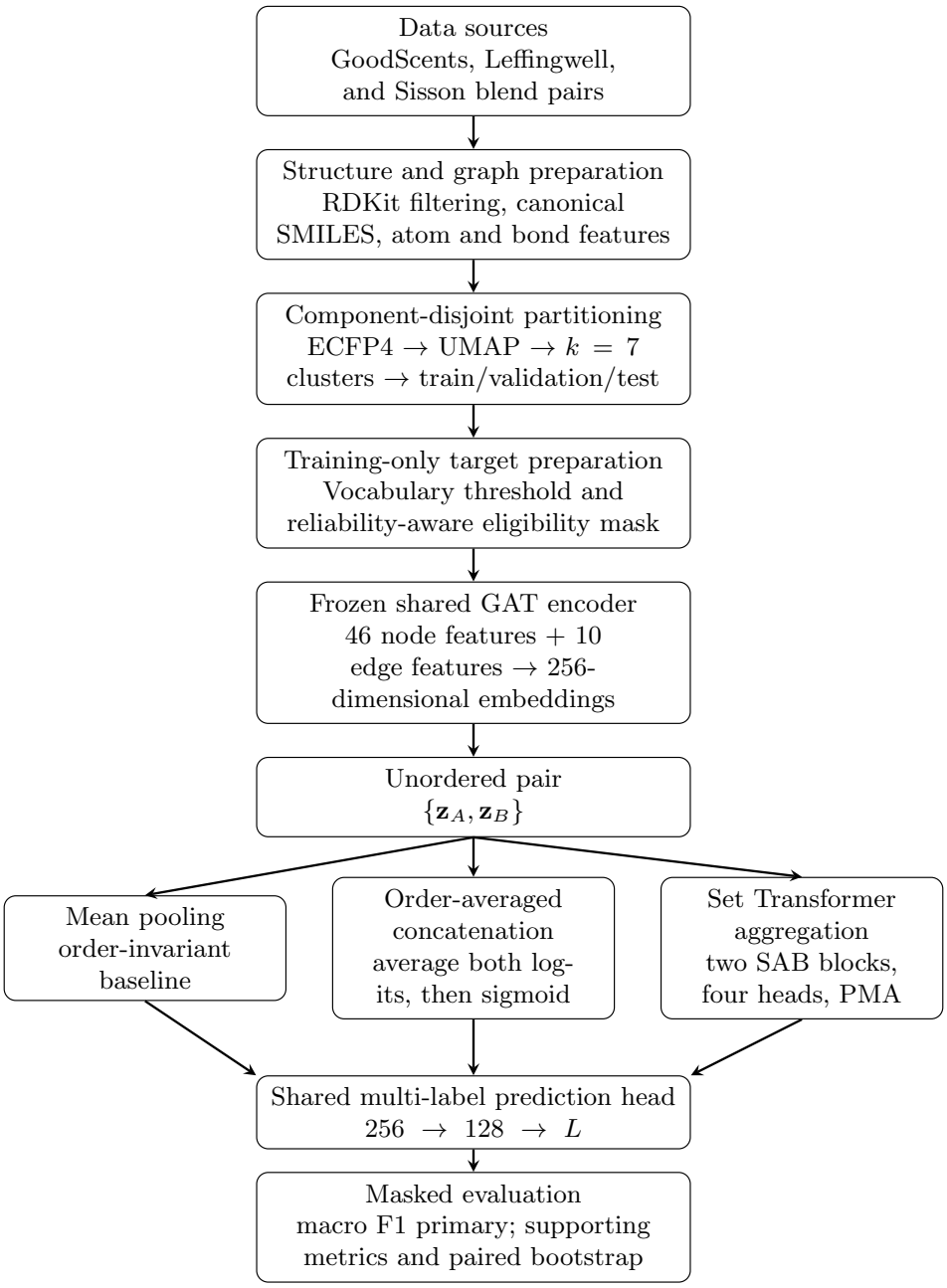


Figure 4.1: Compact overview of the proposed computational pipeline. The three aggregation branches receive the same frozen molecular embeddings and use the same prediction and evaluation protocol.

et al. [37], obtained from their public repository.

Each Pyrfume source has three files: `behavior.csv`, which holds stimulus–descriptor pairs; `stimuli.csv`, which holds the stimulus identifier, the GoodScents or Leffingwell identifier, the PubChem CID, concentration, and solvent; and `molecules.csv`, which holds the CID, molecular weight, isomeric SMILES, IUPAC name, and name. Descriptors in `behavior.csv` are indexed only by stimulus, so they will first be joined to `stimuli.csv` on the stimulus field to obtain a CID, and then to `molecules.csv` on the CID. Concentration and solvent columns will be dropped, because concentration is outside the scope of this study.

Structure filtering. Before molecules are matched across sources, structures will be sanitized with RDKit: salts are removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I are excluded, and molecules outside the range 20–600 g/mol are excluded. The number of molecules retained by these rules will be reported after the finalized pipeline is run. This step uses structure only; no label is consulted.

Join key. The two single-molecule catalogs will be merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both sources, and in the blend dataset, will be canonicalized with RDKit so that the same molecule written in two different ways is matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard and would need further standardization, whereas SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, will be linked to the master dictionary by canonicalizing their SMILES in the same way.

Blend records. The blend dataset of Sisson et al. was built by crawling blender recommendations from GoodScents molecule pages [37]. Each dataset record links a source molecule, a recommended partner, and the catalog-recorded descriptor supplied with that recommendation. Because the two molecule pages were crawled separately, the same unordered pair and descriptor may occur in both directions. During reconstruction, this study will retain the number of directed records, since a count of two provides the reciprocal evidence used in Section 4.3.

4.2.2 Catalog Targets and Their Limits

Catalog-recorded descriptors serve as proxy targets for supervised learning. Reliability procedures estimate the consistency of those records; they do not establish perceptual correctness. The catalogs are curated reference records, not live sensory panel measurements, and their descriptors come from predefined word lists without a reported procedure for combining individual judgments. They therefore contain some noise and subjectivity [6]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

The source data provide molecule identities and catalog descriptors but no reaction conditions or measurements. The computational task therefore treats each record as an unordered pair for label prediction. It does not infer whether a chemical reaction occurs or model a physical interaction mechanism.

4.2.3 Target Vocabulary

Two vocabularies must be kept apart. The *standardized vocabulary* is the list of descriptor names after spelling variants have been merged. The *final target vocabulary*, of size L, is the subset of the standardized vocabulary on which models are actually trained and scored.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same descriptor (for example, anisic and anise, wine and winey, or caramel and caramellic). Reviewing every label by hand would be impractical, so the study will adopt the open-source canonicalization dictionary of Sisson et al. [37]. Their published blend dataset reports 104 standardized olfactory notes. The larger raw vocabulary in the single-molecule catalogs is a different stage of processing and is not treated as an alternative count for the blend labels. Any descriptor in the merged data that the dictionary does not cover will be reviewed manually and either mapped to an existing descriptor or retained as a separately documented descriptor, followed by whitespace and punctuation normalization. Appendix C will list the resulting standardized vocabulary.

Final target vocabulary. The final target vocabulary will be built from *training data only* (Section 4.5.4). Validation and test labels will not be consulted when deciding which descriptors are kept. The size L is therefore not known yet and is written as a symbol throughout. The figure 74

reported by Sisson et al. after their own filtering is not assumed to be this study's L.

Inputs and targets. Input preparation (canonical SMILES to molecular graph with atom and bond features, Section 4.4.1) uses only molecular structure. Target preparation (descriptor standardization, mask construction, vocabulary selection) uses only descriptor labels. Neither step uses the test partition to make a decision.

4.3 Label Reliability Mask

Not every recorded label is equally trustworthy, and not every missing label is a true absence. Rather than assigning one reliability grade to a whole blend, the study will decide, for each *blend–descriptor cell*, whether that cell is included in the stricter eligibility-mask condition. A cell is one entry of the blend-by-descriptor label matrix: one blend and one descriptor.

4.3.1 Consistency Evidence

Two kinds of evidence are used. Both measure consistency, not correctness.

- **Reciprocal-positive evidence (cell level).** A recorded positive is *reciprocal* if the pair was listed under that descriptor from the pages of both molecules (a count of two). The reciprocal and one-sided counts will be recomputed after the finalized cleaning pipeline and reported as dataset characteristics. A one-sided listing is not necessarily wrong, since catalogs do not list every possible blend, but it is not independently confirmed.

- **Descriptor-level consistency (descriptor level).** Agreement between GoodScents and Leffingwell will be measured separately for each descriptor using Cohen's kappa. To keep test information out of this decision, the statistic will be computed only on training-partition molecules that occur in both catalogs. A descriptor with $\kappa \geq 0.21$ will be called *consistent*. The cutoff is an operational agreement rule; reaching it does not prove that the labels are correct.

4.3.2 Eligibility Mask

Table 4.2 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and descriptor j in the eligibility-mask condition.

Table 4.2: Cell-level rules for the eligibility mask.

Catalog status	**Descriptor κ**	**Reciprocal evidence**	**Interpretation**	**Mask**
Listed	≥ 0.21	Yes	Reliable positive	$M_{ij} = 1$
Listed	< 0.21	Yes	Agreement is too weak	$M_{ij} = 0$
Listed	≥ 0.21	No	Not a reliable positive	$M_{ij} = 0$
Unlisted	≥ 0.21	Not applicable	Eligible negative under the closed-world assumption	$M_{ij} = 1$
Unlisted	< 0.21	Not applicable	Negative status is not sufficiently reliable	$M_{ij} = 0$

Here, κ is computed at the descriptor level, whereas M_{ij} is the cell-level decision for blend i and descriptor j. Thus, kappa provides evidence for the rule, while the mask records whether that particular cell is used for training or evaluation.

Because eligible unlisted descriptors enter as negatives, an eligibility-masked test set is not positive-only: a model that predicts a descriptor that is not recorded for a blend is still penalized as a false positive.

Closed-world assumption. Treating an unlisted descriptor as a negative is an operational assumption. It is needed because supervised multi-label learning requires a definition of "absent." Catalog omission may not mean true sensory absence, since a catalog may simply not list a blend or a descriptor. The reliability mask reduces this uncertainty but does not remove it, and results will be interpreted with this limitation in mind.

4.3.3 Experimental Label Conditions

Two label conditions are used.

- **Full-label condition.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed descriptors as positives and unlisted descriptors as negatives.

- **Eligibility-mask condition.** Only cells with $M_{ij} = 1$ under Table 4.2 are used.

Masked loss. In the eligibility-mask condition a blend is kept as long as it has at least one eligible cell; it is not removed merely because some of its cells are ambiguous. Loss is computed only over eligible cells:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i \sum_j M_{ij}\, \ell_{ij}}{\sum_i \sum_j M_{ij}} \tag{4.1}$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). In the full-label condition $M_{ij} = 1$ everywhere and the expression reduces to the ordinary mean loss. The same mask is applied when eligibility-masked metrics are computed (Section 4.7). The training, validation, and test partitions each carry their own mask, built from the same rule.

Insufficient support. After masking, a descriptor may have no eligible positives, or no eligible negatives, in a partition. F1 and PR-AUC are undefined for such a descriptor. Any descriptor with fewer than five eligible positives or fewer than five eligible negatives in the evaluated partition will be excluded from macro-averaged scores, and the excluded descriptors will be listed in the report. A training run that has no eligible cells at all is reported as a failed run and is not silently dropped.

4.4 Model Architecture

All three models share one molecular graph encoder and one output head. They differ in the step that combines the two molecule embeddings. The data flow is: each molecule is turned into a graph and encoded into a vector; the two vectors are combined by the aggregation method; and the combined vector is converted into one probability per descriptor.

4.4.1 Shared Graph Encoder

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a fixed-length vector $\mathbf{z} \in \mathbb{R}^d$ with $d = 256$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$. The same encoder, with the same weights, is applied to both molecules.

The encoder receives the 46-dimensional atom features and 10-dimensional bond features listed in Appendix A. It uses three edge-aware GAT layers. The first two layers use four attention heads

with 128 features per head, and the final layer uses one head to produce a 256-dimensional node representation. A residual connection is used in the second layer, followed by global attention readout to obtain a molecule embedding $\mathbf{z} \in \mathbb{R}^{256}$. The attention computation is defined in Section 3.1.3.

Pretraining. The encoder will first be pretrained on single-molecule data, using a temporary head with one sigmoid output per standardized single-molecule descriptor and binary cross-entropy loss. Only molecules assigned to the training partition (Section 4.5) will be used, so molecules reserved for validation and testing are never seen during pretraining. After pretraining the temporary head is discarded and only the encoder weights are kept.

Encoder policy. The pretrained encoder will be frozen during the primary blend-level comparison. All three variants will therefore receive embeddings produced by the same encoder weights, and only the aggregation module and output head will be trained. This keeps the comparison focused on pair representation rather than allowing the three encoders to diverge during fine-tuning.

4.4.2 Aggregation Variants

Table 4.3 summarizes the three variants. For two molecules, the embeddings form an unordered pair $\{\mathbf{z}_A, \mathbf{z}_B\}$, and a blend prediction should not change if the two molecules are swapped (permutation invariance).

Table 4.3: The three aggregation variants.

Variant	**General operation**	**Order handling**
Mean pooling	Average the two molecule embeddings	Invariant by construction
Order-averaged concatenation	Run both ordered concatenations through the head and average the outputs	Explicit symmetrization
Set Transformer aggregation	Apply set attention, then invariant pooling	Invariant set processing

Mean pooling. The blend embedding is the unweighted average of the two molecule embeddings,

$$\mathbf{z}_{\text{blend}} = \tfrac{1}{2}\left(\mathbf{z}_A + \mathbf{z}_B\right) \in \mathbb{R}^d. \tag{4.2}$$

This is simple and invariant to order. Sum pooling is not included as a separate variant: for exactly two molecules the sum is twice the mean, so it adds no information once the output head is trainable.

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A \,|\, \mathbf{z}_B] \in \mathbb{R}^{2d}$. A single ordered concatenation depends on the order of its inputs, so the model is evaluated on both (A, B) and (B, A) and the two logits are averaged before one sigmoid is applied. This follows the pairwise design of Sisson et al. [37], but it is an *adapted baseline*, not a reproduction of their whole model: it uses this study's encoder, loss, and partitions.

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through a Set Attention Block (SAB), in which each embedding attends to both embeddings in the set (itself included):

$$\mathrm{SAB}(X) = \mathrm{MultiHead}(X, X, X), \tag{4.3}$$

where queries, keys, and values all come from the same set X. This lets the representation of each molecule be adjusted using the other before the two are combined. The attention mechanism follows the definition in Section 3.1.4 of Chapter 3. The adjusted pair is then reduced to a single blend embedding $\mathbf{z}_{\mathrm{blend}} \in \mathbb{R}^d$ by Pooling by Multihead Attention (PMA), which uses learned attention weights instead of a plain average. The aggregator uses two SAB blocks with four heads, a 512-unit row-wise feedforward layer, and PMA with one learned seed vector. Both SAB and PMA are permutation-invariant in the sense needed here: swapping $\mathbf{z}_A$ and $\mathbf{z}_B$ does not change $\mathbf{z}_{\mathrm{blend}}$. This module is a learned way of mixing two embeddings. This study does not claim that it models physical or receptor-level interaction between the molecules.

Complexity. Once the models are implemented, the study will report the number of trainable parameters of each full model and a measure of computation cost (for example, training time per epoch and inference time per batch on the same hardware), so that any accuracy difference can be read together with the cost of obtaining it. These values will be computed from the finalized implementations and reported with the results.

4.4.3 Output Head and Loss

The blend embedding is passed through a Multi-Layer Perceptron that maps $256 \to 128 \to L$, where L is the final target vocabulary size (Section 4.2.3). For order-averaged concatenation the first layer takes 512 inputs. The output layer applies an independent sigmoid to each of the L nodes, because odor descriptors are multi-label: each descriptor is its own present/absent decision, unlike softmax,

which forces the outputs to sum to one. The framework for multi-label learning and the Asymmetric Focal Loss are defined in Section 3.3.3 of Chapter 3; the loss is given in Equation 3.9. Following Hatami and Moradi [18], the loss uses $\gamma_p = 0$ for positive examples and $\gamma_n = 2$ for negative examples. The per-cell loss ℓ_{ij} of this function enters the masked loss of Section 4.3. All three variants use the same loss, optimizer, learning-rate schedule, early-stopping rule, and training budget.

4.5 Component-Disjoint Partitioning and Graph Carving

This section describes how the data are divided into training, validation, and test sets. The aim is that no molecule that appears in a training blend also appears in a test blend. This section describes the procedure; it does not claim that the test set contains only chemically unfamiliar molecules.

Why random pair splitting leaks components. A blend dataset is a collection of pairs, and the same molecule appears in many pairs. If pairs are split at random, a molecule seen in a training pair is very likely to appear again in a test pair, so test performance partly measures memorization of molecules rather than prediction for new ones. Group-aware splitting is the standard remedy for this kind of overlap [22] (see Section 2.5).

4.5.1 Scaffold Split Decay

Scaffold splitting, which groups molecules by shared core framework, is often used for this purpose. In sparse chemical spaces, however, most scaffold groups contain very few molecules, and scaffold splitting then behaves much like random splitting (Section 2.5). To check this on the present data, the molecule-to-scaffold ratio will be computed:

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}} \tag{4.4}$$

where $N_{\text{molecules}}$ is the number of unique canonical molecules and $N_{\text{scaffolds}}$ is the number of distinct Bemis–Murcko frameworks produced by RDKit. A value of R near 1 means almost every scaffold holds a single molecule. R will be computed on the merged master dictionary and compared with the collapse threshold $R < 2.0$ reported by Fooladi et al. [12].

4.5.2 Cluster-Based Partitions

Molecules will be grouped by structural similarity and whole groups will be assigned to one partition. Each unique molecule will be represented by a 2048-bit binary ECFP4 fingerprint with radius 2. UMAP will project the fingerprints to two dimensions using Jaccard distance, 15 neighbors, a minimum distance of 0.1, and random seed 42. K-Means will then form $k = 7$ clusters using 20 initializations and random seed 42. The number of clusters is fixed at seven, following Guo et al. [15].

Clusters will be assigned to training, validation, and test partitions using their molecule counts only. All assignments that give at least one cluster to each partition will be enumerated, and the assignment minimizing the total absolute deviation from a 70:15:15 molecule ratio will be selected. Ties will be resolved lexicographically by cluster identifier. The rule does not inspect descriptor identities, label frequencies, pair counts, or test labels. Once the partitions are fixed they will not be changed; molecule counts, retained pairs, and descriptor support will be reported afterward as outcomes rather than used as selection criteria.

4.5.3 The Graph-Carving Protocol

The blend dataset is treated as a graph in which each node is a unique canonical molecule and each edge is a documented blend between two molecules. Every molecule is assigned to a partition by its cluster. Carving then examines each edge: an edge (blend) is *retained only if both molecules belong to the same assigned partition*, and discarded otherwise. After carving, the retained partitions are component-disjoint, reducing the risk that a molecule component is repeated across training, validation, and test. Sisson et al. [37] applied a similar carving to a random 50:50 molecule partition, which discarded about 83,000 pairs and left about 44,000 training and 40,000 testing pairs. This study uses clusters instead of a random molecule split, so its pair counts will differ and are reported only after the procedure is run. The retained and discarded pair counts for every partition will be reported with the final dataset characteristics.

The validation set is used for early stopping, hyperparameter decisions, and selection of decision thresholds. The test set is kept sealed and used only in the final evaluation.

Component-disjoint partitioning reduces component and structural overlap between partitions. It does not guarantee that test molecules belong to unfamiliar chemical families, because clusters

that are near each other on the map can still contain similar molecules.

4.5.4 Training-Only Vocabulary Filter

Carving changes how often each descriptor occurs, so the final target vocabulary is chosen after partitioning, from training data only. A standardized descriptor d is kept if it has enough positive examples to learn from:

$$d \in \mathcal{D}_{\text{target}} \iff c_{\text{train}}(d) \geq x \tag{4.5}$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive. The threshold is fixed at $x = 30$ training positives. This vocabulary is established once from the full-label training partition and is shared by the full-label and eligibility-mask conditions, so changing the mask does not change the output task. The size of $\mathcal{D}_{\text{target}}$ is L. After the vocabulary is locked, the number of validation and test positives and negatives for each descriptor is *reported* but is not used to add or remove descriptors. Descriptors that end up with too little test support are handled by the insufficient-support rule of Section 4.3 and are listed in the report.

4.6 Experiments

All experiments hold the partitions, the encoder initialization policy, the loss, the optimizer, the stopping rule, the training budget, and the metric suite constant across the three aggregation methods. They will show whether attention-based aggregation performs better than simpler alternatives; a finding that a simpler baseline performs as well or better is a valid and informative result and is reported as such. Each configuration is run with the same five random seeds, $\{0, 1, 2, 3, 4\}$, for every aggregation method.

4.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, using all cells. Experiment B trains each model in the eligibility-mask condition, using masked loss over eligible cells (Section 4.3). Both are evaluated on the held-out test partition. Experiment A is also scored on all cells, and Experiment B is scored on eligible cells with the same mask. Reporting both makes visible how much of any difference depends on which cells are counted.

4.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels will be corrupted on purpose, in the training partition only. Validation and test labels are never changed.

For a noise level $p \in \{0, 0.10, 0.20, 0.30\}$, a fraction p of the eligible positive cells in the training labels is flipped to absent, and an equal number of eligible negative cells is flipped to present, which keeps the density of positives unchanged. Only eligible cells can be flipped, so the same mask logic as Section 4.3 applies. For each combination of noise level and random seed, one corruption mask is generated and *reused across all three aggregation methods*; a new mask is generated for every seed and noise level. The run at $p = 0$ is the uncorrupted eligibility-mask run of Experiment B and is reused. Models are retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is then compared across methods using the paired bootstrap of Section 4.8.

Random flipping tests robustness to label errors in general. It does not reproduce every real process by which catalog errors arise.

4.6.3 Blend-Type Analysis (RQ2)

To see whether the relative performance of the methods differs by catalog-label pattern, each test blend is compared with the recorded descriptors of its two parent molecules. The terms below describe patterns in the catalog records; they are not claims about what a person would perceive.

- **Catalog-label loss:** a descriptor recorded for either parent is not recorded for the blend.
- **Catalog-label gain:** the blend has a recorded descriptor that is not recorded for either parent.
- **Additive:** the blend shows neither loss nor gain.

Descriptive statistics of the label matrix (label density, common co-occurrences, and the rate at which a blend's labels are additive with respect to its parents) are reported for the dataset as a whole. They are used to describe the data and to support this subgroup analysis; they are not used to filter data. The sizes of these three groups are reported and the model comparison is repeated within each group. Because these groups overlap in how they are defined and the analysis is split into subgroups, it is treated as a secondary subgroup and error analysis rather than a confirmatory

test. If any group contains fewer than 5% or more than 95% of the test blends, blends are instead divided by the overlap between the blend descriptors and the union of the parents' descriptors.

4.7 Evaluation Metrics

This section defines the metrics and explains why each is needed when most label cells are negative.

4.7.1 Multi-Label Metrics

Let TP be a descriptor correctly predicted present, FP a descriptor incorrectly predicted present, FN a present descriptor that was missed, and TN a descriptor correctly predicted absent. Most cells in the label matrix are negative, so a model can look accurate by predicting "absent" every time. Precision, recall, and F_1 use only TP, FP, and FN [30], so they ignore the many easy true negatives, and a model that predicts nothing receives $F_1 = 0$.

- **Macro F1 (primary metric).** F_1 is computed for each descriptor separately and then averaged with equal weight, so rare descriptors count as much as common ones [30]. This choice is declared before experimentation and must be the same in Chapters 1–4.
- **Micro F1.** TP, FP, and FN are pooled across all descriptors before F_1 is computed; it summarizes overall performance and favors common descriptors.
- **PR-AUC.** The area under the precision–recall curve, which measures how well the model ranks true descriptors above false ones across all thresholds. Both macro-averaged and micro-averaged PR-AUC will be reported as supporting metrics.
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly (below).

Thresholds. Each descriptor's probability is turned into a present/absent prediction with a decision threshold. For each model and label condition, one global threshold will be selected on validation data by maximizing macro F1 over the grid $\{0.05, 0.10, \ldots, 0.95\}$. The selected threshold will then be applied unchanged to the corresponding test predictions. A global threshold is used to avoid fitting unstable descriptor-specific thresholds for rare labels. Threshold selection matters for imbalanced multi-label problems [45].

4.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). For macro metrics, descriptors with insufficient eligible support are excluded and listed (Section 4.3). With a mask, Hamming loss is

$$\mathrm{HL}_M = \frac{\sum_{i=1}^{N} \sum_{j=1}^{L} M_{ij}\, \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^{N} \sum_{j=1}^{L} M_{ij}} \tag{4.6}$$

where N is the number of evaluated blends, L is the number of target descriptors, y_{ij} is the label, $\hat{y}_{ij} \in \{0, 1\}$ is the thresholded prediction, and $\mathbf{1}(\cdot)$ equals 1 if its argument is true and 0 otherwise. Smaller values are better, with $\mathrm{HL}_M = 0$ for a perfect match. In the full-label condition $M_{ij} = 1$ everywhere and this is the ordinary Hamming loss.

All-negative reference. Hamming loss also rewards the many easy negatives, so it cannot be read alone. The *evaluated-cell density* is

$$D_M = \frac{\sum_i \sum_j M_{ij}\, y_{ij}}{\sum_i \sum_j M_{ij}} \tag{4.7}$$

the fraction of evaluated cells that are positive. A model that predicts every cell absent has $\mathrm{HL}_M = D_M$, which serves as the all-negative reference. It is computed separately for each partition (D_{train}, D_{val}, D_{test}) and for each mask, and a value from one setting is never reused in another. The resulting densities and all-negative references will be reported after the finalized pipeline is run. Every model's Hamming loss is reported next to this reference.

4.8 Statistical Comparison

Differences between models could reflect chance in which blends happen to be in the test set, or chance in training. These two sources are handled separately.

Variability across seeds. Optimization variability comes from random initialization and data order. Each model is trained with the same set of random seeds, and results are summarized across seeds (mean and spread) separately from the bootstrap below.

Bootstrap over blends. Descriptor outcomes within one blend are correlated, so individual cells are not independent. The test blends are therefore resampled as whole rows: a sampled blend keeps all of its descriptor cells together, with its mask. Naive paired t-tests are not appropriate for comparing machine-learning models, because they are sensitive to outliers and not valid when resampled test sets are used [30], and bootstrap resampling shows both whether a difference exists and how large it is [5].

From the N test blends, N rows will be drawn with replacement 10,000 times, using bootstrap seed 42. All models being compared are scored on the *same* resampled indices, giving a paired comparison. For replicate b the paired difference is

$$\Delta(b) = \text{Metric}_{\text{model 1}}(b) - \text{Metric}_{\text{model 2}}(b). \tag{4.8}$$

The direction of a difference is stated with each result: higher is better for macro and micro F1 and for PR-AUC, and lower is better for Hamming loss, so a positive Δ favors model 1 for the first three and model 2 for Hamming loss. The comparison is repeated for each metric and for each pair of aggregation methods (three pairs). Confirmatory inference is limited to the three macro-F1 comparisons. For each pair, the two-sided bootstrap probability is twice the smaller of the proportions of $\Delta(b)$ values at or below zero and at or above zero, using a plus-one correction and capping the result at one. These three probabilities are adjusted with the Holm procedure. The remaining metrics are reported as supporting evidence with effect sizes and confidence intervals, without separate claims of statistical significance.

Intervals and effect size. The family-wise significance level for the confirmatory comparisons is $\alpha = 0.05$ after Holm adjustment. A 95% percentile confidence interval is taken from the 2.5th and 97.5th percentiles of the 10,000 differences and is reported as an effect-size interval rather than used as a separate significance test. The effect size is the mean paired difference, expressed in the units of each metric. For Experiment C, the bootstrap is applied to the change in macro F1 between noise levels.

4.9 Reporting Conventions

Experiments that have not been run are described in the future tense. Any pipeline demonstration run before the final protocol is fixed will be labelled preliminary and will not be reported as a result.

Output from the earlier notebook is not reported as a result of the revised study. The study does not assume that the Set Transformer will outperform the baselines; if mean pooling or order-averaged concatenation matches or exceeds it, that outcome will be reported and discussed on equal footing.

Appendix A

Chemical Feature Vectors

The exact composition of the 46-dimensional node feature vector and the 10-dimensional edge feature vector used for the intra-molecular GAT encoder are detailed below.

Chemical Property	**Dimensions**	**Mathematical / One-Hot Representation & Details**
Atomic Identity	9	One-hot encoding of key organic and halogen elements in flavor/fragrance compounds: [C, N, O, S, F, Cl, Br, I, Other].
Formal Charge	1	Scalar representation of the localized net electrical charge on the atom.
Atomic Degree	6	One-hot representation of the number of connected covalent bonds to heavy atoms: [0, 1, 2, 3, 4, 5+].
Explicit Valence	7	One-hot encoding of the total valence electrons committed to bonding in the molecule: [0, 1, 2, 3, 4, 5, 6+].
Hybridization State	6	One-hot representation of the orbital configuration: [sp, sp^2, sp^3, sp^3d, sp^3d^2, Other].
Aromaticity	1	Binary flag indicating if the atom is part of a resonance-stabilized aromatic ring structure.
Attached Hydrogens	5	One-hot encoding of the number of hydrogens bound to this heavy atom: [0, 1, 2, 3, 4+].
Ring Size Participation	6	Binary flags representing specific ring sizes to map cyclic functional groups: [3-ring, 4-ring, 5-ring, 6-ring, 7-ring, 8-ring].
Ring Membership	1	Binary flag indicating whether the atom is enclosed within any ring structure.
Chirality Tag	4	One-hot representation of tetrahedral stereocenter configuration: [Unspecified, Clockwise (CW), Counterclockwise (CCW), Other].

Table A.1: Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).

Chemical Property	**Dimensions**	**Mathematical / One-Hot Representation & Details**
Bond Type	4	One-hot representation of the bond's electronic sharing structure: [Single, Double, Triple, Aromatic].
Conjugation	1	Binary flag indicating if the bond belongs to a continuous, overlapping pi-electron system.
Ring Membership	1	Binary flag indicating if the bond is closed within a ring structure.
Stereochemical Config	4	One-hot representation of spatial configuration around double bonds: [None, Stereo-Z, Stereo-E, Other].

Table A.2: Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).

A.1 Symmetry Collapse: Representation-Theoretic Derivation

This appendix provides the fuller representation-theoretic argument for a possible expressivity limitation of linear, order-independent readouts at the mixture level, summarised in the mean-pooling discussion of the main text.

Consider a molecular graph G whose set of node embeddings M occupies a d-dimensional representation space $\mathcal{U}$. The structural symmetries of the graph are captured by its automorphism group $\mathcal{H}$, which collects all permutations of the node indices that leave the graph structure unchanged. When a linear, $\mathcal{H}$-invariant readout function f is applied to aggregate these embeddings, representation theory requires that it factor through the group-averaging Reynolds operator p_{avg}, defined as [41]:

$$p_{\text{avg}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \rho(h). \tag{A.1}$$

The Reynolds operator projects the full representation space $\mathcal{U}$ onto its trivial, symmetry-fixed subspace $\mathcal{U}_1$. As a consequence, any linear $\mathcal{H}$-invariant map decomposes into a downstream linear map composed with this group-averaging projection [41]:

$$f = f' \circ p_{\text{avg}}. \tag{A.2}$$

This factorization establishes that a linear readout can only operate on the information that survives in $\mathcal{U}_1$. The non-trivial, symmetry-sensitive components $\mathcal{U}_2 \oplus \cdots \oplus \mathcal{U}_m$, which encode the node-to-node variation responsible for structural differences between molecules, can be discarded by the projection [41]. Thus, two distinct combinations that share the same element-wise sum or mean can become indistinguishable to a downstream classifier. This is a possible information loss in the representation; it does not show that such collisions dominate the present dataset. The result therefore motivates testing attention-based aggregation alongside mean pooling, not replacing mean pooling or assuming that attention will perform better [37, 41].

A.2 Dataset Profiles and Canonical Vocabulary

A.2.1 Single-Molecule Harmonized Dataset Sample

Table A.3 provides representative single-molecule entries resulting from the consolidation of the GoodScents and Leffingwell databases under RDKit organic sanitization and canonicalization.

Table A.3: Representative sample of harmonized single odorants.

CID	Common Name	Canonical SMILES	Harmonized Descriptors
1183	Vanillin	`COc1cc(C=O)ccc1O`	chocolate, creamy, sweet, vanilla
1254	Menthol	`CC1CCC(C(C)C)C(O)C1`	cooling, mentholic, minty, woody
3314	Eugenol	`C=CCc1ccc(O)c(OC)c1`	cinnamon, clove, phenolic, spicy, woody
10582	Myrtenol	`CC1(C)C2CC=C(CO)C1C2`	balsamic, camphoreous, herbal, pine
31276	Isoamyl acetate	`CC(=O)OCCC(C)C`	banana, estery, fruity, pear, sweet
87630	2-Decen-1-ol	`CCCCCCCC=CCO`	citrus, fatty, floral, rose, waxy
637511	Cinnamaldehyde	`O=C/C=C/c1ccccc1`	candy, cinnamon, spicy, warm

A.2.2 Pairwise Blend Mixture Sample

Table A.4 presents sample binary mixtures from the pairwise blend benchmark dataset compiled by Sisson et al. [37], illustrating the constituent molecules and their catalog-recorded blend descriptors.

Table A.4: Sample pairwise odor mixtures from the Sisson et al. benchmark with catalog-recorded descriptors.

Blend	Component 1 (CID)	Component 2 (CID)	Catalog-Recorded Descriptors
B-01	Acetaldehyde (177)	2-Heptanol (10976)	citrus, fruity, pungent
B-02	Acetoin (179)	Bornylcyclohexanol (111394)	buttery, creamy, woody
B-03	2-Oxobutanoic acid (58)	Methyl dihydrojasmonate (102861)	brown, caramellic, floral
B-04	2-Oxobutanoic acid (58)	γ-Undecalactone (7714)	brown, caramellic, fruity
B-05	Pentopyranose (229)	Perillaldehyde (16441)	herbal, spicy
B-06	2-Oxobutanoic acid (58)	Butyric acid (264)	cheesy, sour
B-07	Cinnamaldehyde (637511)	Eugenol (3314)	spicy

A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate

Table A.5 enumerates the complete 104-descriptor standardized vocabulary gate enforced during Phase 1 preprocessing.

Table A.5: The 104 standardized odor descriptors (alphabetical).

1. acetic	27. clean	53. honey	79. potato
2. acidic	28. cocoa	54. jammy	80. powdery
3. alcoholic	29. coconut	55. juicy	81. pungent
4. aldehydic	30. coffee	56. lactonic	82. roasted
5. alliaceous	31. cooling	57. leathery	83. rooty
6. amber	32. corn	58. licorice	84. rummy
7. ammoniacal	33. coumarinic	59. malty	85. salty
8. animal	34. creamy	60. marine	86. smoky
9. anise	35. dairy	61. meaty	87. soapy
10. aromatic	36. dusty	62. medicinal	88. solvent
11. balsamic	37. earthy	63. melon	89. sour
12. berry	38. eggy	64. mentholic	90. spicy
13. bitter	39. estery	65. minty	91. sulfurous
14. bready	40. ethereal	66. moldy	92. sweet
15. brown	41. fatty	67. mossy	93. tarragon
16. burnt	42. fermented	68. mushroom	94. thujonic
17. buttery	43. fishy	69. musk	95. toasted
18. cabbage	44. floral	70. mustard	96. tobacco
19. camphoreous	45. fresh	71. musty	97. tomato
20. caramellic	46. fruity	72. nutty	98. tonka
21. celery	47. fungal	73. oily	99. tropical
22. cheesy	48. fusel	74. onion	100. vanilla
23. chemical	49. garlic	75. orris	101. vegetable
24. cherry	50. green	76. peach	102. waxy
25. chocolate	51. hay	77. phenolic	103. winey
26. citrus	52. herbal	78. pine	104. woody

REFERENCES

[1] R. Achebouche, A. Tromelin, K. Audouze, and O. Taboureau, *Application of artificial intelligence to decode the relationships between smell, olfactory receptors and small molecules*, Scientific Reports, 12 (2022), p. 18817.

[2] R. L. Almeida, V. G. Maltarollo, and F. G. F. Coelho, *Overcoming class imbalance in drug discovery problems: Graph neural networks and balancing approaches*, Journal of Molecular Graphics and Modelling, 126 (2024), p. 108627.

[3] D. Ameta, S. Kumar, R. Mishra, L. Behera, A. Chakraborty, and T. Sandhan, *Odor classification: Exploring feature performance and imbalanced data learning techniques*, PLOS ONE, 20 (2025), p. e0322514.

[4] A. R. N. Aouichaoui, F. Fan, S. S. Mansouri, J. Abildskov, and G. Sin, *Combining group-contribution concept and graph neural networks toward interpretable molecular property models*, Journal of Chemical Information and Modeling, 63 (2023), pp. 725–744.

[5] Y. Bestgen, *Please, don't forget the difference and the confidence interval when seeking for the state-of-the-art status*, in Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, June 2022, European Language Resources Association, pp. 5956–5962.

[6] A. L. Bierling, A. Croy, T. Jesgarzewsky, M. Rommel, G. Cuniberti, T. Hummel, and I. Croy, *A dataset of laymen olfactory perception for 74 mono-molecular odors*, Scientific Data, 12 (2025), p. 347.

[7] J. B. Castro, T. J. Gould, R. Pellegrino, Z. Liang, L. A. Coleman, F. Patel, D. S. Wallace, T. Bhatnagar, J. D. Mainland, and R. C. Gerkin, *Pyrfume: A window to the world's olfactory data*, bioRxiv, (2022).

[8] S. Chithrananda, J. Amores, and K. K. Yang, *Mapping the combinatorial coding between olfactory receptors and perception with deep learning*, bioRxiv, (2026).

[9] G. Coureaud, T. Thomas-Danguin, J.-C. Sandoz, and D. A. Wilson, *Biological constraints on configural odour mixture perception*, J Exp Biol, 225 (2022).

[10] R. Datta, J. Nistane, A. Sose, H. Sahu, and R. Ramprasad, *Machine learning for green solvents: Assessment, selection and substitution*, Advanced Science, (2025).

[11] M. Fey and J. E. Lenssen, *Fast graph representation learning with pytorch geometric*, CoRR, abs/1903.02428 (2019).

[12] H. Fooladi, T. N. L. Vu, M. Mathea, and J. Kirchmair, *Evaluating machine learning models for molecular property prediction: Performance and robustness on out-of-distribution data*, Journal of Chemical Information and Modeling, 65 (2025), pp. 9871–9891.

[13] Y. Fukutani, M. Abe, H. Saito, R. Eguchi, T. Tazawa, C. A. de March, M. Yohda, and H. Matsunami, *Antagonistic interactions between odorants alter human odor perception*, Current Biology, 33 (2023), pp. 2235–2245.e4. doi: 10.1016/j.cub.2023.04.072.

[14] D. Gasser, S. Singh, J. Freiherr, and A. Grasskamp, *Owsum: algorithmic odor prediction and insight into structure-odor relationships*, Journal of Cheminformatics, 15 (2023).

[15] Q. Guo, S. Hernandez-Hernandez, and P. J. Ballester, *Umap-based clustering split for rigorous evaluation of ai models for virtual screening on cancer cell lines**, Journal of Cheminformatics, 17 (2025), p. 94.

[16] R. Hagan, D. Moore, F. McGlone, and S. C. Walker, *Olfactory scene analysis: Does analytical visual processing predict superior identification of component odours in a complex mixture?*, Perception, 54 (2025), pp. 408–430. PMID: 40165414.

[17] S. Han, H. Fu, Y. Wu, G. Zhao, Z. Song, F. Huang, Z. Zhang, S. Liu, and W. Zhang, *Himgnn: a novel hierarchical molecular graph representation learning framework for property prediction*, Briefings in Bioinformatics, 24 (2023), p. bbad305.

[18] F. Hatami and M. Moradi, *Asymmetric focal loss improves graph neural network prediction of drug-drug interactions*, 2026.

[19] Y. P. Heng, H. Y. Lee, J. W. Chong, R. R. Tan, K. B. Aviso, and N. G. Chemmangattuvalappil, *Incorporating machine learning in computer-aided molecular design for fragrance molecules*, Processes, 10 (2022).

[20] B. Hu, H. Zheng, H. Liang, H. Lv, Q. Xiao, Y. Jiang, Q. Lu, Y. Niu, H. Wang, and Z. Xiao, *Elucidating odorant masking effects in food matrices through olfactory receptor-based profiling*, bioRxiv, (2026).

[21] J. Huang, J. Lin, R. Yueng, S. Wu, L. Solla, and T. Acree, *Masking effects on isovaleric acid recognition by sub-threshold odor mixture*, Chemical Senses, 48 (2023), p. bjad047.

[22] R. Joeres, D. B. Blumenthal, and O. V. Kalinina, *Data splitting to avoid information leakage with datasail*, Nature Communications, 16 (2025), p. 3337.

[23] B. K. Lee, E. J. Mayhew, B. Sanchez-Lengeling, J. N. Wei, W. W. Qian, K. A. Little, M. Andres, B. B. Nguyen, T. Moloy, J. Yasonik, J. K. Parker, R. C. Gerkin, J. D. Mainland, and A. B. Wiltschko, *A principal odor map unifies diverse tasks in olfactory perception*, Science, 381 (2023), pp. 999–1006.

[24] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, *Set transformer: A framework for attention-based permutation-invariant neural networks*, 2019.

[25] S. Li, X. Xia, S. Ge, and T. Liu, *Selection-based pseudo-labeling for multi-label learning with noisy labels: Estimating the noise transition*, in Advances in Neural Information Processing Systems, 2022.

[26] E. Makri, N. Nakis, L. Sisson, G. Minsky, L. Tassiulas, V. Satarifard, and N. A. Christakis, *Benchmark for assessing olfactory perception of large language models*, 2026.

[27] A. Paul, *Dextrosinistral reading of smiles notation: Investigation into origin of non-sense code from string manipulations*, Digital Chemical Engineering, 15 (2025), p. 100222.

[28] R. Pellegrino, E. J. Mayhew, J. Margolis, M. Andres, A. B. Wiltschko, R. C. Gerkin, and J. D. Mainland, *Odors smell like their components: A linear framework for predicting olfactory mixture perception*, bioRxiv, (2026).

[29] S. Qin, S. Jiang, J. Li, P. Balaprakash, R. C. Van Lehn, and V. M. Zavala, *Capturing molecular interactions in graph neural networks: a case study in multi-component phase equilibrium*, Digital Discovery, 2 (2023), pp. 138–151.

[30] O. Rainio, J. Teuho, and R. Klén, *Evaluation metrics and statistical tests for machine learning*, Scientific Reports, 14 (2024), p. 6086.

[31] B. C. L. Rodrigues, V. V. Santana, S. Murins, and I. B. R. Nogueira, *Molecule generation and optimization for efficient fragrance creation*, Industrial & Engineering Chemistry Research, 63 (2024), pp. 14480–14494.

[32] S. Romagny, G. Coureaud, and T. Thomas-Danguin, *Developmental switch from elemental to configural processing of odor mixtures in humans*, Developmental Psychobiology, 68 (2026), p. e70188.

[33] K. Saini and V. Ramanathan, *Predicting odor from molecular structure: a multi-label classification approach*, Scientific Reports, 12 (2022), p. 13863.

[34] A. Sajan, S. Sluis, R. Haydarlou, S. Abeln, P. Lisena, R. Troncy, C. Verbeek, I. Leemans, and H. Mouhib, *Hierarchies of smell: Structuring the molecular odor space using semantic taxonomies and machine learning*, Chemical Senses, 51 (2026), p. bjag020.

[35] E. I. Sanchez Medina, S. Linke, M. Stoll, and K. Sundmacher, *Gibbsâ€“helmholtz graph neural network: capturing the temperature dependency of activity coefficients at infinite dilution*, Digital Discovery, 2 (2023), pp. 781–798.

[36] H. Shimakawa, A. Kumada, and M. Sato, *Extrapolative prediction of small-data molecular property using quantum mechanics-assisted machine learning*, npj Computational Materials, 10 (2024), p. 11.

[37] L. Sisson, A. A. Barsainyan, M. Sharma, and R. Kumar, *Deep learning for odor prediction on aroma-chemical blends*, ACS Omega, 10 (2025), pp. 8980–8992.

[38] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, *Learning from noisy labels with deep neural networks: A survey*, IEEE Transactions on Neural Networks and Learning Systems, 34 (2023), pp. 8135–8153.

[39] S. Steshin, *Lo-hi: Practical ml drug discovery benchmark*, 2023.

[40] J. Suh, Y. Hong, and C. Park, *A comparative study of machine learning models on molecular fingerprints for odor decoding*, Communications Chemistry, 8 (2025), p. 278.

[41] M. Talhi, A. Wolf, and A. Monod, *Breaking symmetry bottlenecks in gnn readouts*, 2026.

[42] A. N. Tarekegn, M. Giacobini, and K. Michalak, *A review of methods for imbalanced multi-label classification*, Pattern Recognition, 118 (2021), p. 107965.

[43] G. Tom, C. T. Ser, E. Rajaonson, S. Lo, H. S. Park, B. Lee, and B. Sanchez-Lengeling, *Does this smell the same? learning representations of olfactory mixtures using inductive biases*, Machine Learning: Science and Technology, 6 (2025).

[44] A. Vanacore and M. S. Pellegrino, *Benchmarking procedures for characterizing the extent of rater agreement: A comparative study*, Quality and Reliability Engineering International, 38 (2022), pp. 1404–1415.

[45] T. Wen, X. Cai, and J. Li, *Graph neural networks vs. traditional qsar: A comprehensive comparison for multi-label molecular odor prediction*, Molecules, 30 (2025).

[46] D. S. Wigh, J. M. Goodman, and A. A. Lapkin, *A review of molecular representation in the age of machine learning*, WIREs Computational Molecular Science, 12 (2022), p. e1603.

[47] Y.-X. Wu, X. Wang, A. Zhang, X. He, and T.-S. Chua, *Discovering invariant rationales for graph neural networks*, 2022.

[48] Z. Wu, J. Wang, H. Du, D. Jiang, Y. Kang, D. Li, P. Pan, Y. Deng, D. Cao, C.-Y. Hsieh, and T. Hou, *Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking*, Nature Communications, 14 (2023), p. 2585.

[49] H. Xie, J. Sun, Y. Shao, S. Li, S. Hou, Y. Sun, and J. Wang, *Molecular odor prediction based on multi-feature graph attention networks*, 2025.

[50] L. Xu, D.-J. Zou, and S. Firestein, *Odor mixtures: A chord with silent notes*, Frontiers in Ecology and Evolution, Volume 11 - 2023 (2023).

[51] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, *Deep sets*, 2018.

[52] M. Zhang, Y. Hiki, A. Funahashi, and T. J. Kobayashi, *A deep position-encoding model for predicting olfactory perception from molecular structures and electrostatics*, NPJ systems biology and applications, 10 (2024), p. 76.

[53] W. Zhu, Y. Zhang, D. Zhao, J. Xu, and L. Wang, *Hignn: Hierarchical informative graph neural networks for molecular property prediction equipped with feature-wise attention*, 2022.